User Manual

for S32K3 I3C Driver

Document Number: UM34I3CASR4.4 Rev0000R2.0.0 Rev. 1.0

1 Revision History 2

2 Introduction 3

2.1 Supported Derivatives . . . 3
2.2 Overview . . . 4
2.3 About This Manual . . . 4
2.4 Acronyms and Definitions . . . 5
2.5 Reference List . . . 5

3 Driver 6

3.1 Requirements . . . 6
3.2 Driver Design Summary . . . 6
3.3 Hardware Resources . . . 7
3.4 Deviations from Requirements . . . 7
3.5 Driver Limitations . . . 7
3.6 Driver usage and configuration tips . . . 8
3.6.1 I3c initialization . . . 8
3.6.2 I3c functions common to both master and slave . . . 8
3.6.3 I3c Master . . . 8
3.6.4 I3c Slave . . . 9
3.6.5 I3c Dynamic Address Assignment . . . 9
3.6.6 I3c In-Band Interrupts . . . 10
3.6.7 I3C Hot-Join . . . 10
3.7 Runtime errors . . . 11
3.8 Symbolic Names Disclaimer . . . 11

4 Tresos Configuration Plug-in 12

4.1 Module I3c . . . 14
4.2 Container GeneralConfiguration . . . 14
4.3 Parameter I3cEnableSlaveRequests . . . 15
4.4 Parameter I3cEnableI3cfeatures . . . 15
4.5 Parameter I3cInitDynamicProcessAssignment . . . 16
4.6 Parameter I3cDevErrorDetect . . . 16
4.7 Parameter I3cDmaFeature . . . 16
4.8 Parameter I3cMulticoreSupport . . . 17
4.9 Parameter I3cEnableUserModeSupport . . . 17
4.10 Parameter I3cVersionInfoApi . . . 18
4.11 Parameter I3cTimeoutMethod . . . 18
4.12 Parameter I3cTimeoutVal . . . 20
4.13 Container I3cChannel . . . 20
4.14 Parameter I3cChannelId . . . 21

4.15 Parameter I3cHwChannel . . . 21
4.16 Parameter I3cMasterSlaveMode . . . 22
4.17 Container I3cMasterConfiguration . . . 22
4.18 Parameter I3cMasterEnable . . . 23
4.19 Parameter I3cDisableTimeout . . . 23
4.20 Parameter I3cMasterHighKeeper . . . 24
4.21 Parameter I3cI2cBaud . . . 24
4.22 Parameter I3cOpenDrainBaud . . . 25
4.23 Parameter I3cPushPullBaud . . . 26
4.24 Parameter I3cPushPullLow . . . 27
4.25 Parameter I3cOpenDrainHighPP . . . 27
4.26 Parameter I3cOpenDrainStop . . . 28
4.27 Parameter I3cPpBaudRate . . . 28
4.28 Parameter I3cOdBaudRate . . . 29
4.29 Parameter I3cI2cBaudRate . . . 29
4.30 Parameter I3cSkew . . . 30
4.31 Parameter I3cMasterTransferType . . . 30
4.32 Parameter I3cIbiResp . . . 31
4.33 Parameter I3cSendStop . . . 32
4.34 Parameter I3cEmitAutoIbi . . . 32
4.35 Parameter I3cEnableStartOptimization . . . 33
4.36 Parameter I3cNoMandatoryByte . . . 33
4.37 Parameter I3cMasterCallback . . . 33
4.38 Parameter I3cMasterGetIbiAddressCallback . . . 34
4.39 Reference I3cMasterClockReference . . . 35
4.40 Reference I3cMasterDmaTxChannel . . . 35
4.41 Reference I3cMasterDmaRxChannel . . . 35
4.42 Container I3cIbiAddressRegistry . . . 36
4.43 Parameter I3cSlaveAddress . . . 36
4.44 Container I3cDynamicAddressList . . . 37
4.45 Parameter I3cSlaveDynamicAddress . . . 37
4.46 Container I3cSlaveConfiguration . . . 38
4.47 Parameter I3cSlaveEnable . . . 38
4.48 Parameter I3cNack . . . 39
4.49 Parameter I3cMatchStartStop . . . 39
4.50 Parameter I3cIgnoreS0S1Error . . . 40
4.51 Parameter I3cHighDataRateSupport . . . 40
4.52 Parameter I3cOfflineSupport . . . 40
4.53 Parameter I3cBusAvailableMatch . . . 41
4.54 Parameter I3cStaticAddress . . . 41

4.55 Parameter I3cSlaveTransferType . . . 42
4.56 Parameter I3cTransferSize . . . 42
4.57 Parameter I3cBCR . . . 43
4.58 Parameter I3cDCR . . . 43
4.59 Parameter I3cVendorId . . . 44
4.60 Parameter I3cPartNumber . . . 44
4.61 Parameter I3cVendorInfo . . . 45
4.62 Parameter I3cActState . . . 45
4.63 Parameter I3cPendingInt . . . 46
4.64 Parameter I3cExtendedIbiData . . . 47
4.65 Parameter I3cCountExtraBytes . . . 47
4.66 Parameter I3cMapIndex . . . 47
4.67 Parameter I3cClockFrequency . . . 48
4.68 Parameter I3cClockAccuracy . . . 48
4.69 Parameter I3cHotJoinDevice . . . 49
4.70 Parameter I3cSlaveCallback . . . 49
4.71 Reference I3cSlaveDmaTxChannel . . . 50
4.72 Reference I3cSlaveDmaRxChannel . . . 50
4.73 Container CommonPublishedInformation . . . 51
4.74 Parameter ArReleaseMajorVersion . . . 51
4.75 Parameter ArReleaseMinorVersion . . . 52
4.76 Parameter ArReleaseRevisionVersion . . . 52
4.77 Parameter ModuleId . . . 52
4.78 Parameter SwMajorVersion . . . 53
4.79 Parameter SwMinorVersion . . . 53
4.80 Parameter SwPatchVersion . . . 54
4.81 Parameter VendorApiInfix . . . 54
4.82 Parameter VendorId . . . 55

5 Module Index **56**
5.1 Software Specification . . . 56

6 Module Documentation **57**
6.1 I3c . . . 57
6.1.1 Detailed Description . . . 57
6.1.2 Data Structure Documentation . . . 61
6.1.3 Macro Definition Documentation . . . 62
6.1.4 Types Reference . . . 84
6.1.5 Enum Reference . . . 85
6.1.6 Function Reference . . . 88
6.2 I3c_ip . . . 93

6.2.1 Detailed Description 93
6.2.2 Data Structure Documentation 97
6.2.3 Macro Definition Documentation 101
6.2.4 Types Reference 122
6.2.5 Enum Reference 122
6.2.6 Function Reference 127

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	31.03.2022	NXP RTD Team	Prepared for release RTD S32K3 2.0.0

Chapter 2

Introduction

- Supported Derivatives
- Overview
- About This Manual
- Acronyms and Definitions
- Reference List

This User Manual describes NXP Semiconductor I3C for S32K3XX. I3C driver configuration parameters and deviations from the specification are described in Driver chapter of this document. I3C driver requirements and APIs are described in the I3C driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k344_mapbga257
- s32k344_mqfp172
- s32k324_mapbga257
- s32k324_mqfp172
- s32k314_mapbga257
- s32k314_mqfp172
- s32k312_mqfp172
- s32k312_mqfp100
- s32k322_mqfp172
- s32k322_mqfp100
- s32k342_mqfp172
- s32k342_mqfp100
- s32k341_mqfp172
- s32k341_mqfp100

All of the above microcontroller devices are collectively named as S32K3.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CDD	Complex Device Driver
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
I3C	Improved Inter-Integrated Circuit
SIU	Systems Integration Unit
SWS	Software Specification
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	S32K3XX Reference Manual	Rev. 3, 10/2021
2	S32K3XX Datasheet	Rev. 3, 10/2021
3	Errata S32K312 (0P09C)	Rev. 10/Jan/2022
4	Errata S32K342 (0P97C)	Rev. 13 Dec 2021
5	Errata S32K344 (0P55A/1P55A)	Rev. 30 Sept 2021

Chapter 3

Driver

- Requirements
- Driver Design Summary
- Hardware Resources
- Deviations from Requirements
- Driver Limitations
- Driver usage and configuration tips
- Runtime errors
- Symbolic Names Disclaimer

3.1 Requirements

I3c is a Complex Device Driver (CDD), so there are no AUTOSAR requirements regarding this module.

It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

The MIPI Alliance Improved Inter-Integrated Circuit (MIPI I3C) brings major improvements in use and power over I2C, and provides an alternative to SPI for mid-speed applications. The I3C bus protocol supports:

- In-band interrupts: interrupts can go from Slave to Master without extra wires, such that the Master knows which Slave sent the interrupt.
- In-band Common Command Codes (CCC)
- Dynamic addressing
- Multi-master / multi-drop
- Hot-Join(HJ)

- SDR / HDR-DDR transfers
- I2C compatibility The I3C peripheral supports all required and most optional features of the MIPI Alliance Specification for I3C, v1.0, except for the ternary data rates (HDR-TSP, HDR-TSL).

The driver uses the I3C peripheral in S32K3XX microcontrollers which has one available instance. The driver can be used to configure the I3C peripheral and initiate synchronous and asynchronous data transfers. The peripheral can be configured to operate in slave and master mode. Hardware and software settings can be configured using an Autosar standard configuration tool. The information required for a I3c data transfer will be configured in a data structure that will be sent as parameter to the API of the driver.

3.3 Hardware Resources

The I3c driver uses the I3C hardware IP. For more details and availability please check the device reference manual.

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR CDD_I3C Driver software specification in some places. The table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, not available, not testable or out of scope for the I3C Driver.

Term	Definition
N/A	Not available
N/T	Not testable
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, not available, not testable or out of scope for the driver.

3.5 Driver Limitations

I3c driver limitations:

- I3C driver has not been validated for SCL >6MHz.
- Master Request is not supported.
- SDR and HDR-DDR read/write option 2 method is not supported.
- CCC commands have not been validated with HDR-DDR mode.
- Dynamic address assignment with DCR=1 is not working.

- Manual ACK/NACK decision for In-Band Interrupts is not working.
- HDR-DDR sync master write not working for buffer sizes larger than 16.
- Missing DMA support for master TX, RX, slave RX.
- HDR-DDR sync master read misses last byte.
- InBandInterrupt address is always 0 in Get Ibi Address Callback when DynamicAddressAssignment is done at Init.

3.6 Driver usage and configuration tips

3.6.1 I3c initialization

- In order to use the I3c driver, each logical channel which maps a hardware unit must be first initialized using I3c_Init() function.
- Each logical channel is initialized in one of the three modes: master, slave, or master capable (acts like a slave after initialization, but will be prepared to switch to master mode).
- Once a channel is initialized, it cannot be initialized again until it is de-initialized using I3c_DeInit() function.

3.6.2 I3c functions common to both master and slave

- The I3c_GetTransferStatus() function is used to get the status of a transfer.
- The I3c_GetVersionInfo() function is used to get the version information of the I3c module.
- The I3c_GetError() function is used to get the entire error/warning register value for the current I3c channel.
- The I3c_GetStatus() function is used to get the entire status register value for the I3c channel.

3.6.3 I3c Master

- The I3c_SyncTransfer() function is used to initiate a blocking transfer on the I3c Bus.
- The I3c_AsyncTransfer() function is used to initiate a non-blocking transfer on the I3c Bus.

- The I3c_AssignDynamicAddress() function is used to assign dynamic addresses to the slave devices which do not have one assigned.
- The I3c_MasterSetBaudRate() function is used to set the Push-Pull, Open-Drain, and I2c baud rates for the I3c channel configured as a master.
- The I3c_MasterGetBaudRate() function is used to get the Push-Pull, Open-Drain, and I2c baud rates of the I3c channel.
- The I3c_MasterSetIbiOptions() function is used to set the In-Band Interrupt options for the following Slave Requests.
- The I3c_MasterManualIbiResponse() function is used to ACK/NACK an In-Band Interrupt when manual intervention is needed.

3.6.4 I3c Slave

- The I3c_PrepareSlaveBuffer() function is used to prepare either Tx or Rx data buffer for a slave device.
- The I3c_GetDynamicAddressInfo() function is used to get the slave's dynamic address when it was assigned, re-assigned, or reset.
- The I3c_PrepareSlaveTransmission() function is used to fill in the slave Tx FIFO with data, before the master requests a read from the slave. I3c_PrepareSlaveBuffer() should be called before.
- The I3c_SlaveRequestEvent() function is used to request an In-Band Interrupt, Hot-Join, or Master Request.

3.6.5 I3c Dynamic Address Assignment

- At each I3c bus initializtion and after a new Slave device is added to the bus (through Hot-Join mechanism), the master shall assign all devices a Dynamic Address.
- Once a slave device receives a Dynamic Address, it should be used for all subsequent transactions on the I3c bus, until and unless the master changes this Dynamic Address. The only way a master can change the Dynamic Address is by using RSTDAA CCC command or SETNEWDA CCC command. In addition, these addresses establish the priority for the In-Band Interrupts. The lower the address, the higher the priority of the In-Band Interrupt.
- Dynamic Address Assignment process is initiated using the I3c_AssignDynamicAddress() function. It can be used either with the address list set in configurator, or it can be passed new addresses list at runtime.
- Before Dynamic Address Assignment, master shall first assign dynamic addresses using SETDASA CCC command for I3c slave devices that also have a static address assigned.

3.6.6 I3c In-Band Interrupts

- In order to request an interrupt, an I3c slave device shall emit its Address into the arbitrated Address header following a START (but not following a Repeated START). If no START is forthcoming within the Bus Available Condition, then the I3c slave may issue a START request by pulling the SDA line low and wait for the master to also pull the SCL line low to complete the START signal.

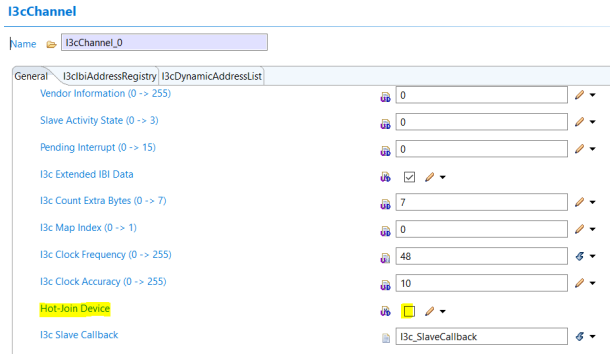
- The master shall pull SCL line low if a new transfer is initiated or if the engine Auto Emits START + 0x7E to allow the In-Band Interrupt to win the arbitration. To specify if the engine Auto Emits START + 0x7E on a future In-Band Interrupt, I3c_MasterSetIbiOptions() function could be used.
- One or more data bytes can follow an accepted In-Band Interrupt. This operation is similar to a read from the slave and all related rules apply.

- An In-Band Interrupt is requested using I3c_SlaveRequestEvent() function.
- I3c_MasterGetIbiAddressCallback callback can be used to get the In-Band Interrupt address. If manual intervention is needed, the decision can be taken depending on this address.

3.6.7 I3C Hot-Join

- Hot-Join is a mechanism which allows Slaves to join the I3c Bus after it has been already configured. Hot-↩ Joining Slaves may be any valid Slave type, including Secondary Master devices.

- To specify if a Slave device is a Hot-Join device, the corresponding box in the configurator shall be checked.



- A Hot-Join request is a particular case of an In-Band Interrupt and could be pushed only after a START. Thus, the slave pushes onto the I3c bus the Reserved Address 0x02 as an IBI with W (write) direction. This address is subject to Address Arbitration.

- As for any In-Band Interrupt, if no START is upcoming on the I3c Bus, the engine could Auto Emit START + 0x7E when the Slave Request is pushed.
  In this way, any Hot-Join Request will win the Address Arbitration and the Slave Device will be allowed to adhere to the I3c Bus.

- To specify if the engine Auto Emits START + 0x7E on a future Slave Request, I3c_MasterSetIbiOptions() function could be used.

- Hot-Join Request always requires Manual intervention to decide if the Master ACKs or NACKs the request. Thus, I3c_MasterManualIbiResponse() function must be called from the provided user callback.

3.7 Runtime errors

None.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like:

#define <Mip>Conf_<Container_ShortName>_<Container_ID>

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module I3c
  - Container GeneralConfiguration
    - Parameter I3cEnableSlaveRequests
    - Parameter I3cEnableI3cfeatures
    - Parameter I3cInitDynamicProcessAssignment
    - Parameter I3cDevErrorDetect
    - Parameter I3cDmaFeature
    - Parameter I3cMulticoreSupport
    - Parameter I3cEnableUserModeSupport
    - Parameter I3cVersionInfoApi
    - Parameter I3cTimeoutMethod
    - Parameter I3cTimeoutVal
  - Container I3cChannel
    - Parameter I3cChannelId
    - Parameter I3cHwChannel
    - Parameter I3cMasterSlaveMode
    - Container I3cMasterConfiguration
      - Parameter I3cMasterEnable
      - Parameter I3cDisableTimeout
      - Parameter I3cMasterHighKeeper
      - Parameter I3cI2cBaud
      - Parameter I3cOpenDrainBaud
      - Parameter I3cPushPullBaud
      - Parameter I3cPushPullLow
      - Parameter I3cOpenDrainHighPP
      - Parameter I3cOpenDrainStop
      - Parameter I3cPpBaudRate
      - Parameter I3cOdBaudRate
      - Parameter I3cI2cBaudRate
      - Parameter I3cSkew

- Parameter I3cMasterTransferType
- Parameter I3cIbiResp
- Parameter I3cSendStop
- Parameter I3cEmitAutoIbi
- Parameter I3cEnableStartOptimization
- Parameter I3cNoMandatoryByte
- Parameter I3cMasterCallback
- Parameter I3cMasterGetIbiAddressCallback
- Reference I3cMasterClockReference
- Reference I3cMasterDmaTxChannel
- Reference I3cMasterDmaRxChannel
- Container I3cIbiAddressRegistry
- Parameter I3cSlaveAddress
- Container I3cDynamicAddressList
- Parameter I3cSlaveDynamicAddress

* Container I3cSlaveConfiguration
  - Parameter I3cSlaveEnable
  - Parameter I3cNack
  - Parameter I3cMatchStartStop
  - Parameter I3cIgnoreS0S1Error
  - Parameter I3cHighDataRateSupport
  - Parameter I3cOfflineSupport
  - Parameter I3cBusAvailableMatch
  - Parameter I3cStaticAddress
  - Parameter I3cSlaveTransferType
  - Parameter I3cTransferSize
  - Parameter I3cBCR
  - Parameter I3cDCR
  - Parameter I3cVendorId
  - Parameter I3cPartNumber
  - Parameter I3cVendorInfo
  - Parameter I3cActState
  - Parameter I3cPendingInt
  - Parameter I3cExtendedIbiData
  - Parameter I3cCountExtraBytes
  - Parameter I3cMapIndex
  - Parameter I3cClockFrequency
  - Parameter I3cClockAccuracy
  - Parameter I3cHotJoinDevice
  - Parameter I3cSlaveCallback
  - Reference I3cSlaveDmaTxChannel
  - Reference I3cSlaveDmaRxChannel

– Container CommonPublishedInformation

* Parameter ArReleaseMajorVersion
* Parameter ArReleaseMinorVersion
* Parameter ArReleaseRevisionVersion
* Parameter ModuleId
* Parameter SwMajorVersion
* Parameter SwMinorVersion
* Parameter SwPatchVersion
* Parameter VendorApiInfix
* Parameter VendorId

4.1 Module I3c

Configuration of the Synchronous Audio Interface (I3c) module.

Included containers:

- GeneralConfiguration
- I3cChannel
- CommonPublishedInformation

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.2 Container GeneralConfiguration

This container contains the global configuration parameters of the Non-Autosar I3c driver.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Parameter I3cEnableSlaveRequests

Enable/Disable Slave Requests(In-Band Interrupt, Hot-Join and Master Request).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.4 Parameter I3cEnableI3cfeatures

Enable/Disable I3c features.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.5 Parameter I3cInitDynamicProcessAssignment

Enable/Disable Dynamic Address Assignment process at Init() function. And this process can be also done at runtime by using API by using

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.6 Parameter I3cDevErrorDetect

Switches the Development Error Detection and Notification ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.7 Parameter I3cDmaFeature

Enable DMA feature for transferring and Notification ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.8 Parameter I3cMulticoreSupport

This parameter globally enables the possibility to support multicore. If I3cMulticoreSupport is disabled, then for all the variants no partition shall be defined. If I3cMulticoreSupport is enabled, at least one EcucPartition needs to be defined (in all variants).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.9 Parameter I3cEnableUserModeSupport

I3c Enable User Mode Support

When this parameter is enabled, the I3c module will adapt to run from User Mode, with

the following measure : configuring REG_PROT for I3c IPs so that the registers under

protection can be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1

for more information and availability on this platform, please see chapter "User Mode Support" in IM.

True: I3c module will adapt to run from User Mode.

False:I3c module will not apdapt to run from User Mode./p>

If this parameter is not ediatable, that means I3c driver can run in User Mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.10 Parameter I3cVersionInfoApi

Adds / removes the service I3c_GetVersionInfo() from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.11 Parameter I3cTimeoutMethod

Configures the timeout method for I3c.

Based on this selection a certain timeout method from OsIf will be used in the driver.

Note: If OSIF_COUNTER_SYSTEM or OSIF_COUNTER_CUSTOM are selected make sure the corresponding timer is enabled in OsIf General configuration.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COU↩ NTER_CUSTOM']

4.12 Parameter I3cTimeoutVal

This is a timeout (microseconds) value which is used to wait for each synchronization transfer

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	30000
max	4294967295
min	1

4.13 Container I3cChannel

This container contains the configuration (parameters) of the I3c Controller(s).

Included subcontainers:

- I3cMasterConfiguration
- I3cSlaveConfiguration

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.14 Parameter I3cChannelId

Identifies the I3c channel.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	0
min	0

4.15 Parameter I3cHwChannel

Selects hardware channel.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	I3C_0
literals	['I3C_0']

4.16 Parameter I3cMasterSlaveMode

Select wether the selected channel will be used as Master, Slave or both.

MASTER: The module is an I3c Master.

SLAVE: The module is an I3c Slave.

MASTER_CAPABLE: The I3C module is master-capable; however the module is operating as a slave now.

When used from the start, the I3C module will start as a slave, but will be prepared to switch to master mode using specific procedures.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	MASTER
literals	['MASTER', 'SLAVE', 'MASTER_CAPABLE']

4.17 Container I3cMasterConfiguration

This container contains the configuration (parameters) of the I3c Master Controller(s).

Included subcontainers:

- I3cIbiAddressRegistry
- I3cDynamicAddressList

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.18 Parameter I3cMasterEnable

Indicates if the master is enabled and what states are allowed for it (the master).

MASTER_OFF: Master is not enabled.

MASTER_ON: Master is enabled.

MASTER_CAPABLE: Master capable, but operating as a slave. Only avaialble when I3c is in MASTER_CAPABLE mode.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	MASTER_ON
literals	['MASTER_OFF', 'MASTER_ON', 'MASTER_CAPABLE']

4.19 Parameter I3cDisableTimeout

Disable timeout to prevent application errors (MERRWARN).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.20 Parameter I3cMasterHighKeeper

Indicates how High-Keeper is to be supported.

HIGH_KEEPER_NONE: Use PUR to hold SCL high.

HIGH_KEEPER_WIRED_IN: Use pin_HK controls.

PASSIVE_SDA: Passive on SDA: can Hi-Z for Bus Free and hold.

PASSIVE_SDA_SCL: Passive on SDA and SCL: can Hi-Z both for Bus Free, and can Hi-Z SDA for hold.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	HIGH_KEEPER_NONE
literals	['HIGH_KEEPER_NONE', 'HIGH_KEEPER_WIRED_IN', 'PASSIVE_S↩ DA', 'PASSIVE_SDA_SCL']

4.21 Parameter I3cI2cBaud

I3c baud count in cycles.

Indicates how many ODBAUD periods are needed to communicate with I2C devices.

For example, if ODBAUD gives 200 ns, and the goal is Fm+ (Fast Mode, 1 MHz), then the sum needs to be 1 us.

If odd vs. even, then I2CBAUD acts differently. For example, if I2CBAUD = 3, it gives 3 ODBAUD periods low and 2 ODBAUD periods high.

If I2CBAUD = 4, then it gives 4 ODBAUD periods for low and 4 ODBAUD periods for high.

Also, if I2CBAUD = 3, this yields 200 * 3 = 600 ns and 200 * 2 = 400 ns, with the sum 600 + 400 = 1000 ns = 1us.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	10
max	15
min	0

4.22 Parameter I3cOpenDrainBaud

I3c open-drain baud count in cycles.

The number of PPBAUD periods to make up one I3C Open-Drain half-clock baseline.

ODBAUD half-clock time period = ODBAUD * PPBAUD high period. (if ODHPP is 0)

Formula for SCL in Open-Drain mode is (if ODHPP=0)

SCL(in MHz) = FCLK / [(2 + 2 * PPBAUD) * (ODBAUD + 1)]

Target to get open-drain half clock period is 200ns.

For example:

If PPBAUD yields 12.5 MHz (40 ns per PPBAUD period), then ODBAUD = 5 can be

used to get 200ns. See also ODHPP for details on short High and long Low.

If PPBAUD = 3, FCLK = 120MHz, ODBAUD = 5 then open-drain SCL = 120/(2 + 2*3)*(5+1) = 2.5 MHz.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	255
min	0

4.23 Parameter I3cPushPullBaud

I3c push-pull baud count in cycles.

This sets the Push-Pull frequency as a divider from the FCLK.

This sets the SCL half-clock period baseline (used at least for the high time of SCL).

Formula for SCL in Push-Pull mode using PPBAUD and PPLOW is:

SCL (Freq in MHz) = FCLK / ((2 + 2 * PPBAUD) + PPLOW).

If PPLOW is 0, then SCL high = SCL low (in ns) = (1 + PPBAUD) * 1000 /FCLK(MHz)

Examples:

If FCLK = 24 MHz, then PPBAUD = 0 yields 12 MHz (42.67 ns per half period)

If FCLK = 50 MHz, then PPBAUD = 1 yields 12.5 MHz (20 + 20 = 40 ns per half period)

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	11
max	15
min	0

4.24 Parameter I3cPushPullLow

Changes the duty cycle for Push-Pull. It indicates how many more FCLKs to use for low.

Formula for SCL in Push-Pull mode using PPBAUD and PPLOW is:

SCL(Freq in MHz) = FCLK / ((2 + 2 * PPBAUD) + PPLOW)

If PPLOW is non-zero, then SCL High (in ns) = [(1 + PPBAUD) * 1000] / FCLK(MHz)

SCL low (in ns) = [(1 + PPBAUD + PPLOW) * 1000] / FCLK(MHz)

For example, a 50 MHz FCLK, a PPBAUD of 1, and a PPLOW of 1, yields

[(1 + 1) * 1000] / 50 = 20 + 20 = 40 ns high and [(1 + 1 + 1) * 1000] / 50 = 20 + 20 + 20 = 60 ns low. This would be

equivalent of 10 MHz SCL timing, but maintaining the 40 ns high needed, so that I2C devices do not see the high periods.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.25 Parameter I3cOpenDrainHighPP

Optional field that allows for the I3c open drain to be long low and short high.

The High period of SCL is the PPBAUD period. This leaves enough time for the pull-up resistor to pull

the SDA high when SCL is low, but is quick when SCL is high and there are no changes happening.

ODBAUD low half-clock time period = ODBAUD * PPBAUD high period.

ODBAUD high half-clock time period = PPBAUD high period

Formula for SCL in Open-Drain mode is (if ODHPP = 1):

SCL(in MHz) = FCLK /[(1 + PPBAUD) * (ODBAUD + 1) + (1 + PBAUD)]

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.26 Parameter I3cOpenDrainStop

If true, then STOP will be emitted at open-drain speeds even for I3c messages.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.27 Parameter I3cPpBaudRate

I3c Push-Pull Baud Rate (Hz).

When not in I2c mode, the I3c bus is always initialized in SDR mode. This value is the Push-Pull frequency (in Hz) for the SDR mode.

For HDR-DDR mode, one should use the I3c_MasterSetBaudRate() function to change the Push-Pull frequency accordingly.

Calculated value.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0.0
max	1.29E7
min	0.0

4.28 Parameter I3cOdBaudRate

I3c Open-Drain Baud Rate (Hz). Calculated value.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2500000.0
max	5000000.0
min	0.0

4.29 Parameter I3cI2cBaudRate

I2C Baud Rate (Hz). Calculated value.

Property	Value
type	ECUC-FLOAT-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0.0
max	1200000.0
min	0.0

4.30 Parameter I3cSkew

Number of FCLK counts for an SDA change after SCL for I3c push-pull.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	7
min	0

4.31 Parameter I3cMasterTransferType

Type of the transfer: DMA or interrupts.

DMA: The driver will use DMA to perform a transfer.

INTERRUPTS: The driver will use interrupts to perform a transfer.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	INTERRUPTS
literals	['DMA', 'INTERRUPTS']

4.32 Parameter I3cIbiResp

In-Band Interrupt (IBI) response.

ACK: ACK with no mandatory byte.

NACK: NACK.

ACK_MANDATORY: ACK with mandatory byte.

MANUAL: Reserved.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	ACK
literals	['ACK', 'NACK', 'ACK_MANDATORY', 'MANUAL']

4.33 Parameter I3cSendStop

If this is true, then a STOP is emitted after a Slave Request has been processed.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.34 Parameter I3cEmitAutoIbi

An In-Band interrupt request means that the slave will push the corresponding address
(its Dynamic Address for IBI and Master Request, and the reserved address 0x02 for Hot-Join)
after a START (but not after a Repeated START). If no START is forthcoming on the I3c Bus,
then the slave will pull the SDA line low and will wait for a START. This can happen unexpectedly
(on a START from a transfer for example), or Auto IBI feature can be used. Using Auto IBI feature means that
the master holds in a stopped state and emits START + 0x7E continuously such that it will always
lose the header arbitration, in favor of the interrupt.
If this box is checked, then Auto IBI feature is used for the following In-Band interrupts.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.35 Parameter I3cEnableStartOptimization

If this is true, then all I3c Dynamic Addresses must be less than 0x40. This improves the speed for sending the START header.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.36 Parameter I3cNoMandatoryByte

If true, then the ADDRx from In-Band Interrupt Rules register refer to slaves without mandatory byte.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.37 Parameter I3cMasterCallback

I3c Master Callback. This function will be called for all I3c events.

Property	Value
type	ECUC-FUNCTION-NAME-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.38 Parameter I3cMasterGetIbiAddressCallback

This callback is used by the user to get the address of the slave

which requested an In-Band interrupt. This callback is used only if the

In-Band interrupt won the header arbitration. If the In-Band interrupt requires

manual intervention, based on the slave's address, the user can use MasterCallback for I3C_IP_MASTER_EVENT_IBI event

and the I3c_Ip_MasterManualIbiResponse() function to make the ACK/NACK decision. If no manual intervention is required,

this callback can be used so that the user is aware of the slave's address which requested the IBI.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.39 Reference I3cMasterClockReference

I3c FCLK which is set in Mcu module configuration.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/AUTOSAR/EcucDefs/Mcu/McuModuleConfiguration/McuClockSetting↩ Config/McuClockReferencePoint']

4.40 Reference I3cMasterDmaTxChannel

Specifies DMA Tx channel used.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type']

4.41 Reference I3cMasterDmaRxChannel

Specifies DMA Rx channel used.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type']

4.42 Container I3cIbiAddressRegistry

Addresses of the slaves which use In-Band Interrupts.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	11
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.43 Parameter I3cSlaveAddress

Slave addresses.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	127
min	0

4.44 Container I3cDynamicAddressList

List of the dynamic addresses to be assigned at the I3c Bus initialization.

Note:

The first address will be assigned to the slave with the lowest Provisional ID, and so on.

The dynamic address establish the interrupt priority for the slaves (the lower the address, the higher the priority).

If I3c Start Optimization is enabled, all addresses must be lower than 0x40.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	11
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.45 Parameter I3cSlaveDynamicAddress

Slave dynamic addresses.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	127
min	0

4.46 Container I3cSlaveConfiguration

This container contains the configuration (parameters) of the I3c Slave Controller(s).

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.47 Parameter I3cSlaveEnable

If true, then a slave can operate on the I2c or I3c bus. Otherwise, ignore I2c or I3c bus.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.48 Parameter I3cNack

If true, then the slave will NACK all requests to it, except a CCC broadcast.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.49 Parameter I3cMatchStartStop

Whether to assert START/STOP status only if the slave is addressed.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.50 Parameter I3cIgnoreS0S1Error

Whether to ignore S0/S1 error in SDR mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.51 Parameter I3cHighDataRateSupport

If true, the slave allow HDR-DDR messaging, if available.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.52 Parameter I3cOfflineSupport

Whether to wait 60 us of bus quiet or HDR Exit Pattern to ensure bus is not in HDR mode and safely track the message.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.53 Parameter I3cBusAvailableMatch

Provides the count of the slow clock to count out 1 us (or more), to allow an In-Band Interrupt (IBI) to drive SDA low.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	23
max	63
min	0

4.54 Parameter I3cStaticAddress

Slave Static Address.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	127
min	0

4.55 Parameter I3cSlaveTransferType

Type of the transfer: DMA or interrupts.

DMA: The driver will use DMA to perform a transfer.

INTERRUPTS: The driver will use interrupts to perform a transfer.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	INTERRUPTS
literals	['DMA', 'INTERRUPTS']

4.56 Parameter I3cTransferSize

Transfer size: bytes or half words

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	BYTES
literals	['BYTES', 'HALF_WORDS']

4.57 Parameter I3cBCR

Bus Characteristics.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	102
max	255
min	0

4.58 Parameter I3cDCR

Device Characteristics.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.59 Parameter I3cVendorId

Slave Vendor ID.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	283
max	32767
min	0

4.60 Parameter I3cPartNumber

Slave ID Part Number.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4294967295
min	1

4.61 Parameter I3cVendorInfo

The value will be returned in Vendor Reserved field by the GETSTATUS CCC command.

The vendor information should be maintained by the application, because the master reads this field.

If VENDINFO is not configured, then the GETSTATUS field always returns 0.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.62 Parameter I3cActState

The value will be returned in Activity Mode field by the GETSTATUS CCC command.

The activity state should be maintained by the application, because the master will read

this field. If the activity state is not configured, then the GETSTATUS command will always return 0.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	3
min	0

4.63 Parameter I3cPendingInt

Should be set to the pending interrupt that the GETSTATUS CCC (command code) will return.

The pending interrupt should be maintained by the application, because the master reads this field.

If PENDINT=0 and

? If an IBI interrupt is pending, then the GETSTATUS command returns 1.

? If an IBI interrupt is not pending, then the GETSTATUS field returns 0.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.64 Parameter I3cExtendedIbiData

If true, then there are more bytes following the mandatory byte.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.65 Parameter I3cCountExtraBytes

The number of extra bytes past the Mandatory Data Byte.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	7
min	0

4.66 Parameter I3cMapIndex

Index of Dynamic Address that IBI is for.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	1
min	0

4.67 Parameter I3cClockFrequency

Slave control clock frequency.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	48
max	255
min	0

4.68 Parameter I3cClockAccuracy

Slave control clock accuracy.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	10
max	255
min	0

4.69 Parameter I3cHotJoinDevice

If this is true, then this I3c device is a Hot-Join device and all specific requirements apply.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.70 Parameter I3cSlaveCallback

I3c Slave Callback. This function will be called for all I3c events.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.71 Reference I3cSlaveDmaTxChannel

Specifies DMA Tx channel used.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type']

4.72 Reference I3cSlaveDmaRxChannel

Specifies DMA Rx channel used.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type']

4.73 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.74 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.75 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.76 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.77 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	255
max	255
min	255

4.78 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	2
max	2
min	2

4.79 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.80 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.81 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.82 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

This chapter describes the Tresos configuration plug-in for the I3C Driver. The most of the parameters are described below.

Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

I3c 57
I3c_ip 93

Chapter 6

Module Documentation

6.1 I3c

6.1.1 Detailed Description

Data Structures

- struct I3c_HwChannelConfigType

  This structure contains the Hw Channel Configuration of the I3C module. More...
- struct I3c_HwConfigType

  This structure contains the Hw Unit Configuration of the I3C module. More...
- struct I3c_RequestType

  This structure is passed to I3c_SyncTransmit or I3c_AsyncTransmit functions. It holds the necessary information required for a transfer on the I3C Bus. More...
- struct I3c_ConfigType

  This structure contains initialization data for the I3C driver. More...

Macros

- #define I3C_INIT_ID

  APIs service IDs.
- #define I3C_DEINIT_ID
- #define I3C_SYNCTRANSFER_ID
- #define I3C_ASYNCTRANSFER_ID
- #define I3C_GETTRANSFERSTATUS_ID
- #define I3C_PREPARESLAVEBUFFER_ID
- #define I3C_GETVERSIONINFO_ID
- #define I3C_GETERROR_ID
- #define I3C_PREPARESLAVETXFIFO_ID
- #define I3C_MASTERSETBAUDRATE_ID
- #define I3C_MASTERGETBAUDRATE_ID
- #define I3C_GETSTATUS_ID

- #define I3C_BROADCAST_ADDRESS

  I3c broadcast address.
- #define I3C_MASTER_ERROR_NACK

  I3C master error flags.
- #define I3C_MASTER_ERROR_WRITE_ABORT
- #define I3C_MASTER_ERROR_TERM
- #define I3C_MASTER_ERROR_PARITY
- #define I3C_MASTER_ERROR_CRC
- #define I3C_MASTER_ERROR_OREAD
- #define I3C_MASTER_ERROR_OWRITE
- #define I3C_MASTER_ERROR_MSG
- #define I3C_MASTER_ERROR_INVALID_REQUEST
- #define I3C_MASTER_ERROR_TIMEOUT
- #define I3C_SLAVE_ERROR_RX_OVERRUN

  I3C slave error flags.
- #define I3C_SLAVE_ERROR_TX_UNDERRUN
- #define I3C_SLAVE_ERROR_TX_UNDERRUN_NACK
- #define I3C_SLAVE_ERROR_TERM
- #define I3C_SLAVE_ERROR_INVALID_START
- #define I3C_SLAVE_ERROR_SDR_PARITY
- #define I3C_SLAVE_ERROR_HDR_PARITY_ERROR
- #define I3C_SLAVE_ERROR_HDR_CRC
- #define I3C_SLAVE_ERROR_S0S1
- #define I3C_SLAVE_ERROR_OREAD
- #define I3C_SLAVE_ERROR_OWRITE
- #define I3C_MASTER_STATUS_IBIADDR

  I3C master status flags.
- #define I3C_MASTER_STATUS_NOWMASTER
- #define I3C_MASTER_STATUS_ERROR_WARNING
- #define I3C_MASTER_STATUS_IBIWON
- #define I3C_MASTER_STATUS_TXNOTFULL
- #define I3C_MASTER_STATUS_RXPENDING
- #define I3C_MASTER_STATUS_COMPLETE
- #define I3C_MASTER_STATUS_CTRLDONE
- #define I3C_MASTER_STATUS_SLVSTART
- #define I3C_MASTER_STATUS_IBITYPE
- #define I3C_MASTER_STATUS_NACKED
- #define I3C_MASTER_STATUS_BETWEEN
- #define I3C_MASTER_STATUS_STATE
- #define I3C_SLAVE_STATUS_TIMECTRL

  I3C slave status flags.
- #define I3C_SLAVE_STATUS_ACTSTATE
- #define I3C_SLAVE_STATUS_HJDIS
- #define I3C_SLAVE_STATUS_MRDIS
- #define I3C_SLAVE_STATUS_IBIDIS
- #define I3C_SLAVE_STATUS_EVDET
- #define I3C_SLAVE_STATUS_EVENT
- #define I3C_SLAVE_STATUS_CHANDLED
- #define I3C_SLAVE_STATUS_HDRMATCHED
- #define I3C_SLAVE_STATUS_ERROR_WARNING

- #define I3C_SLAVE_STATUS_CCC
- #define I3C_SLAVE_STATUS_DACHG
- #define I3C_SLAVE_STATUS_TXNOTFULL
- #define I3C_SLAVE_STATUS_RXPENDING
- #define I3C_SLAVE_STATUS_STOP
- #define I3C_SLAVE_STATU_MATCHED
- #define I3C_SLAVE_STATUS_START
- #define I3C_SLAVE_STATUS_STHDR
- #define I3C_SLAVE_STATUS_STDAA
- #define I3C_SLAVE_STATUS_STREQWR
- #define I3C_SLAVE_STATUS_STREQRD
- #define I3C_SLAVE_STATUS_STCCCH
- #define I3C_SLAVE_STATUS_STMSG
- #define I3C_SLAVE_STATUS_STNOTSTOP
- #define I3C_CCC_ENEC_BROADCAST

  Common Command Codes. These defines should be used with I3c_SendCommonCommandCode() and I3c_Read↩ CommonCommandCode(). For more details about each command, please see Mipi Basic Specifications v1.0 document.
- #define I3C_CCC_DISEC_BROADCAST
- #define I3C_CCC_ENTAS0_BROADCAST
- #define I3C_CCC_ENTAS1_BROADCAST
- #define I3C_CCC_ENTAS2_BROADCAST
- #define I3C_CCC_ENTAS3_BROADCAST
- #define I3C_CCC_RSTDAA_BROADCAST
- #define I3C_CCC_ENTDAA_BROADCAST
- #define I3C_CCC_DEFSLVS_BROADCAST
- #define I3C_CCC_SETMWL_BROADCAST
- #define I3C_CCC_SETMRL_BROADCAST
- #define I3C_CCC_ENTTM_BROADCAST
- #define I3C_CCC_ENDXFER_BROADCAST
- #define I3C_CCC_ENTHDR0_BROADCAST
- #define I3C_CCC_ENEC_DIRECT
- #define I3C_CCC_DISEC_DIRECT
- #define I3C_CCC_ENTAS0_DIRECT
- #define I3C_CCC_ENTAS1_DIRECT
- #define I3C_CCC_ENTAS2_DIRECT
- #define I3C_CCC_ENTAS3_DIRECT
- #define I3C_CCC_RSTDAA_DIRECT
- #define I3C_CCC_SETDASA_DIRECT
- #define I3C_CCC_SETNEWDA_DIRECT
- #define I3C_CCC_SETMWL_DIRECT
- #define I3C_CCC_SETMRL_DIRECT
- #define I3C_CCC_GETMWL_DIRECT
- #define I3C_CCC_GETMRL_DIRECT
- #define I3C_CCC_GETPID_DIRECT
- #define I3C_CCC_GETBCR_DIRECT
- #define I3C_CCC_GETDCR_DIRECT
- #define I3C_CCC_GETSTATUS_DIRECT
- #define I3C_CCC_GETACCMST_DIRECT
- #define I3C_CCC_ENDXFER_DIRECT
- #define I3C_CCC_SETBRGTGT_DIRECT
- #define I3C_CCC_GETMXDS_DIRECT

Types Reference

- typedef uint8 I3c_PartCoreType

  In-Band Interrupt (IBI) response.
- typedef uint8 I3c_DataType

  Data Type to be sent or received.
- typedef I3c_Ip_SlaveDeviceType I3c_SlaveDeviceType

  Slave device characteristics structure.
- typedef I3c_Ip_MasterBaudRateType I3c_MasterBaudRateType

  Contains the baud rates options. This structure should be set by the user at runtime.

Enum Reference

- enum I3c_MasterSlaveModeType

  Definition of the I3C Hw Unit mode.
- enum I3c_DirectionType

  Direction of the transfer: send/receive.
- enum I3c_BusType

  Bus type with EmitStartAddr.
- enum I3c_TransferSizeType

  Indicates size of the transfer message.
- enum I3c_StatusType

  I3C specific error codes.
- enum I3c_SlaveRequestType

  Indicates the current slave operation mode.

Function Reference

- void I3c_Init (const I3c_ConfigType *const Config)

  Initialize the I3C module.
- void I3c_DeInit (void)

  Deinitialize the I3C module.
- Std_ReturnType I3c_SyncTransfer (const uint8 Channel, uint8 *DataBuffer, const uint32 BufferSize, const I3c_RequestType *const Request)

  Send or receive a blocking message on the I3C bus.
- Std_ReturnType I3c_AsyncTransfer (const uint8 Channel, uint8 *DataBuffer, const uint32 BufferSize, const I3c_RequestType *const Request)

  Send or receive a non-blocking message on the I3C bus.
- I3c_StatusType I3c_GetTransferStatus (const uint8 Channel, uint32 *const BytesRemaining)

  Get the status of an I3C channel.
- Std_ReturnType I3c_PrepareSlaveBuffer (const uint8 Channel, uint8 *DataBuffer, const uint8 BufferSize)

  Prepare the Tx or Rx data buffer for a slave channel.
- uint32 I3c_GetError (const uint8 Channel)

  Returns the error register, either for the master or the slave, depending on the configured channel.
- Std_ReturnType I3c_PrepareSlaveTransmission (const uint8 Channel)

Enables the slave Tx Send event if used in I2C mode.

- Std_ReturnType I3c_MasterSetBaudRate (const uint8 Channel, const uint32 FunctionalClock, const I3c_MasterBaudRateType *BaudRates, const I3c_BusType BusType)

  Set the baud rate for all subsequent I3C communications.
- Std_ReturnType I3c_MasterGetBaudRate (const uint8 Channel, const uint32 FunctionalClock, I3c_MasterBaudRateType *BaudRates)

  Get the baud rates of the I3C module.
- uint32 I3c_GetStatus (const uint8 Channel)

  Returns the error register, either for the master or the slave, depending on the configured channel.

6.1.2 Data Structure Documentation

6.1.2.1 struct I3c_HwChannelConfigType

This structure contains the Hw Channel Configuration of the I3C module.

Definition at line 401 of file CDD_I3c_Types.h.

6.1.2.2 struct I3c_HwConfigType

This structure contains the Hw Unit Configuration of the I3C module.

Definition at line 411 of file CDD_I3c_Types.h.

Data Fields

Type	Name	Description
const uint8	I3c_HwUnit	I3c hardware instance
const I3c_MasterSlaveModeType	MasterSlaveModeConfig	
const I3c_HwChannelConfigType *	HwChannelConfig	

6.1.2.3 struct I3c_RequestType

This structure is passed to I3c_SyncTransmit or I3c_AsyncTransmit functions. It holds the necessary information required for a transfer on the I3C Bus.

Definition at line 423 of file CDD_I3c_Types.h.

Data Fields

Type	Name	Description
uint8	SlaveAddress	Slave Device Address
boolean	SendStop	Specifies if either a STOP or repeated START signal shall be emitted
I3c_DirectionType	Direction	Specifies the direction of the transfer, send or receive
I3c_TransferSizeType	TransferSize	Specifies if the transfer is in half words or in bytes
I3c_BusType	BusType	Bus Type: SDR, DDR, I2C

6.1.2.4 struct I3c_ConfigType

This structure contains initialization data for the I3C driver.

Definition at line 436 of file CDD_I3c_Types.h.

Data Fields

Type	Name	Description
const I3c_PartCoreType	I3c_CoreId	Numeric Partition Id
const I3c_HwConfigType *const *	I3c_HwConfig	Pointer to the I3C Hw Unit configuration

6.1.3 Macro Definition Documentation

6.1.3.1 I3C_INIT_ID

```
#define I3C_INIT_ID
```

APIs service IDs.

Service IDs for the I3C driver APIs. I3c_Init() ID

Definition at line 136 of file CDD_I3c.h.

6.1.3.2 I3C_DEINIT_ID

```
#define I3C_DEINIT_ID
```

I3c_Deinit() ID

Definition at line 137 of file CDD_I3c.h.

6.1.3.3 I3C_SYNCTRANSFER_ID

```
#define I3C_SYNCTRANSFER_ID
```

I3c_SyncTransmit() ID

Definition at line 138 of file CDD_I3c.h.

6.1.3.4 I3C_ASYNCTRANSFER_ID

```
#define I3C_ASYNCTRANSFER_ID
```

I3c_AsyncTransmit() ID

Definition at line 139 of file CDD_I3c.h.

6.1.3.5 I3C_GETTRANSFERSTATUS_ID

```
#define I3C_GETTRANSFERSTATUS_ID
```

I3c_GetTransferStatus() ID

Definition at line 140 of file CDD_I3c.h.

6.1.3.6 I3C_PREPARESLAVEBUFFER_ID

```
#define I3C_PREPARESLAVEBUFFER_ID
```

I3c_PrepareSlaveBuffer() ID

Definition at line 141 of file CDD_I3c.h.

6.1.3.7 I3C_GETVERSIONINFO_ID

```
#define I3C_GETVERSIONINFO_ID
```

I3c_GetVersionInfo() ID

Definition at line 142 of file CDD_I3c.h.

6.1.3.8 I3C_GETERROR_ID

```
#define I3C_GETERROR_ID
```

I3c_GetError() ID

Definition at line 146 of file CDD_I3c.h.

6.1.3.9 I3C_PREPARESLAVETXFIFO_ID

```
#define I3C_PREPARESLAVETXFIFO_ID
```

I3c_PrepareSlaveTransmission() ID

Definition at line 150 of file CDD_I3c.h.

6.1.3.10 I3C_MASTERSETBAUDRATE_ID

```
#define I3C_MASTERSETBAUDRATE_ID
```

I3c_MasterSetBaudRate() ID

Definition at line 154 of file CDD_I3c.h.

6.1.3.11 I3C_MASTERGETBAUDRATE_ID

```
#define I3C_MASTERGETBAUDRATE_ID
```

I3c_MasterGetBaudRate() ID

Definition at line 155 of file CDD_I3c.h.

6.1.3.12 I3C_GETSTATUS_ID

```
#define I3C_GETSTATUS_ID
```

I3c_GetStatus() ID

Definition at line 160 of file CDD_I3c.h.

6.1.3.13 I3C_BROADCAST_ADDRESS

```
#define I3C_BROADCAST_ADDRESS
```

I3c broadcast address.

All I3c slave devices shall match this address. In addition, no I2c slave device shall match this address because it is reserved and unused in I2c.

Definition at line 116 of file CDD_I3c_Types.h.

6.1.3.14 I3C_MASTER_ERROR_NACK

```
#define I3C_MASTER_ERROR_NACK
```

I3C master error flags.

These can be used with I3c_GetError() function to filter the active errors. Use these masks to check which specific errors/warnings occurred. Slave(s) NACKed the last address

Definition at line 125 of file CDD_I3c_Types.h.

6.1.3.15 I3C_MASTER_ERROR_WRITE_ABORT

```
#define I3C_MASTER_ERROR_WRITE_ABORT
```

Slave NACKed the write data

Definition at line 126 of file CDD_I3c_Types.h.

6.1.3.16 I3C_MASTER_ERROR_TERM

```
#define I3C_MASTER_ERROR_TERM
```

Master terminated a slave read

Definition at line 127 of file CDD_I3c_Types.h.

6.1.3.17 I3C_MASTER_ERROR_PARITY

```
#define I3C_MASTER_ERROR_PARITY
```

Parity error from a DDR read

Definition at line 128 of file CDD_I3c_Types.h.

6.1.3.18 I3C_MASTER_ERROR_CRC

```
#define I3C_MASTER_ERROR_CRC
```

CRC error from a DDR read

Definition at line 129 of file CDD_I3c_Types.h.

6.1.3.19 I3C_MASTER_ERROR_OREAD

```
#define I3C_MASTER_ERROR_OREAD
```

Read from MRDATAB register when FIFO empty

Definition at line 130 of file CDD_I3c_Types.h.

6.1.3.20 I3C_MASTER_ERROR_OWRITE

```
#define I3C_MASTER_ERROR_OWRITE
```

Write to MWDATAB register when FIFO full

Definition at line 131 of file CDD_I3c_Types.h.

6.1.3.21 I3C_MASTER_ERROR_MSG

```
#define I3C_MASTER_ERROR_MSG
```

Read/write SDR/DDR mismatch

Definition at line 132 of file CDD_I3c_Types.h.

6.1.3.22 I3C_MASTER_ERROR_INVALID_REQUEST

```
#define I3C_MASTER_ERROR_INVALID_REQUEST
```

Invalid use of a request

Definition at line 133 of file CDD_I3c_Types.h.

6.1.3.23 I3C_MASTER_ERROR_TIMEOUT

```
#define I3C_MASTER_ERROR_TIMEOUT
```

The module has stalled too long in a frame

Definition at line 134 of file CDD_I3c_Types.h.

6.1.3.24 I3C_SLAVE_ERROR_RX_OVERRUN

```
#define I3C_SLAVE_ERROR_RX_OVERRUN
```

I3C slave error flags.

These can be used with I3c_GetError() function to filter the active errors. Use these masks to check which specific errors/warnings occurred. Slave internal RX buffer/FIFO overrun

Definition at line 143 of file CDD_I3c_Types.h.

6.1.3.25 I3C_SLAVE_ERROR_TX_UNDERRUN

```
#define I3C_SLAVE_ERROR_TX_UNDERRUN
```

Slave internal TX buffer/FIFO underrun

Definition at line 144 of file CDD_I3c_Types.h.

6.1.3.26 I3C_SLAVE_ERROR_TX_UNDERRUN_NACK

```
#define I3C_SLAVE_ERROR_TX_UNDERRUN_NACK
```

The internal Tx buffer/FIFO was underrun in the read header and so the module NACKed the header

Definition at line 145 of file CDD_I3c_Types.h.

6.1.3.27 I3C_SLAVE_ERROR_TERM

```
#define I3C_SLAVE_ERROR_TERM
```

Master terminated a read from slave

Definition at line 146 of file CDD_I3c_Types.h.

6.1.3.28 I3C_SLAVE_ERROR_INVALID_START

```
#define I3C_SLAVE_ERROR_INVALID_START
```

Invalid start with SCL falling while SDA = 1 is in a STOP condition

Definition at line 147 of file CDD_I3c_Types.h.

6.1.3.29 I3C_SLAVE_ERROR_SDR_PARITY

```
#define I3C_SLAVE_ERROR_SDR_PARITY
```

SDR parity error on message from master

Definition at line 148 of file CDD_I3c_Types.h.

6.1.3.30 I3C_SLAVE_ERROR_HDR_PARITY_ERROR

```
#define I3C_SLAVE_ERROR_HDR_PARITY_ERROR
```

HDR parity error on message from master

Definition at line 149 of file CDD_I3c_Types.h.

6.1.3.31 I3C_SLAVE_ERROR_HDR_CRC

```
#define I3C_SLAVE_ERROR_HDR_CRC
```

HDR-DDR CRC error on a message from master

Definition at line 150 of file CDD_I3c_Types.h.

6.1.3.32 I3C_SLAVE_ERROR_S0S1

```
#define I3C_SLAVE_ERROR_S0S1
```

S0 or S1 error

Definition at line 151 of file CDD_I3c_Types.h.

6.1.3.33 I3C_SLAVE_ERROR_OREAD

```
#define I3C_SLAVE_ERROR_OREAD
```

The SRDATAB register was read for more bytes than were available

Definition at line 152 of file CDD_I3c_Types.h.

6.1.3.34 I3C_SLAVE_ERROR_OWRITE

```
#define I3C_SLAVE_ERROR_OWRITE
```

The SWDATA register was written when FULL

Definition at line 153 of file CDD_I3c_Types.h.

6.1.3.35 I3C_MASTER_STATUS_IBIADDR

```
#define I3C_MASTER_STATUS_IBIADDR
```

I3C master status flags.

These can be used with I3c_GetStatus() function to filter the active status flags. Use these masks to check which specific status is set. The address of the Slave Request (IBI, MR, HJ)

Definition at line 162 of file CDD_I3c_Types.h.

6.1.3.36 I3C_MASTER_STATUS_NOWMASTER

```
#define I3C_MASTER_STATUS_NOWMASTER
```

Now this module is a master, but was previously a slave and mastership was handed

Definition at line 163 of file CDD_I3c_Types.h.

6.1.3.37 I3C_MASTER_STATUS_ERROR_WARNING

```
#define I3C_MASTER_STATUS_ERROR_WARNING
```

An error or warning occurred

Definition at line 164 of file CDD_I3c_Types.h.

6.1.3.38 I3C_MASTER_STATUS_IBIWON

```
#define I3C_MASTER_STATUS_IBIWON
```

A slave request won the arbitration on a header address, regardless of whether it was ACKed/NACKed

Definition at line 165 of file CDD_I3c_Types.h.

6.1.3.39 I3C_MASTER_STATUS_TXNOTFULL

```
#define I3C_MASTER_STATUS_TXNOTFULL
```

Tx FIFO not yet full, according to its watermark

Definition at line 166 of file CDD_I3c_Types.h.

6.1.3.40 I3C_MASTER_STATUS_RXPENDING

```
#define I3C_MASTER_STATUS_RXPENDING
```

Rx FIFO has receiving data bytes, according to its watermark

Definition at line 167 of file CDD_I3c_Types.h.

6.1.3.41 I3C_MASTER_STATUS_COMPLETE

```
#define I3C_MASTER_STATUS_COMPLETE
```

A message has completed. For more details, please check the Reference Manual

Definition at line 168 of file CDD_I3c_Types.h.

6.1.3.42 I3C_MASTER_STATUS_CTRLDONE

```
#define I3C_MASTER_STATUS_CTRLDONE
```

A Master Control request was completed. For more details, please check the Reference Manual

Definition at line 169 of file CDD_I3c_Types.h.

6.1.3.43 I3C_MASTER_STATUS_SLVSTART

```
#define I3C_MASTER_STATUS_SLVSTART
```

A slave is/was requesting a START by pulling the SDA low

Definition at line 170 of file CDD_I3c_Types.h.

6.1.3.44 I3C_MASTER_STATUS_IBITYPE

```
#define I3C_MASTER_STATUS_IBITYPE
```

Slave request type (IBI, MR, HJ)

Definition at line 171 of file CDD_I3c_Types.h.

6.1.3.45 I3C_MASTER_STATUS_NACKED

```
#define I3C_MASTER_STATUS_NACKED
```

The last address header was NACKed by the addressed slave

Definition at line 172 of file CDD_I3c_Types.h.

6.1.3.46 I3C_MASTER_STATUS_BETWEEN

```
#define I3C_MASTER_STATUS_BETWEEN
```

Between messages or Dynamic Address Assignment. For more details, please check the Reference Manual

Definition at line 173 of file CDD_I3c_Types.h.

6.1.3.47 I3C_MASTER_STATUS_STATE

```
#define I3C_MASTER_STATUS_STATE
```

Indicates the current master state

Definition at line 174 of file CDD_I3c_Types.h.

6.1.3.48 I3C_SLAVE_STATUS_TIMECTRL

```
#define I3C_SLAVE_STATUS_TIMECTRL
```

I3C slave status flags.

These can be used with I3c_GetStatus() function to filter the active status flags. Use these masks to check which specific status is set. Indicates if time control is currently enabled

Definition at line 189 of file CDD_I3c_Types.h.

6.1.3.49 I3C_SLAVE_STATUS_ACTSTATE

```
#define I3C_SLAVE_STATUS_ACTSTATE
```

Activity state from Common Command Codes (CCC)

Definition at line 190 of file CDD_I3c_Types.h.

6.1.3.50 I3C_SLAVE_STATUS_HJDIS

```
#define I3C_SLAVE_STATUS_HJDIS
```

Hot-Join is disabled at this time. While Hot-Join is disabled, MCTRL requests will be held off

Definition at line 191 of file CDD_I3c_Types.h.

6.1.3.51 I3C_SLAVE_STATUS_MRDIS

```
#define I3C_SLAVE_STATUS_MRDIS
```

Master Requests are disabled at this time. While Master Requests are disabled, MCTRL requests will be held off

Definition at line 192 of file CDD_I3c_Types.h.

6.1.3.52 I3C_SLAVE_STATUS_IBIDIS

```
#define I3C_SLAVE_STATUS_IBIDIS
```

In-Band interrupts are disabled at this time. While Master Requests are disabled, MCTRL requests will be held off

Definition at line 193 of file CDD_I3c_Types.h.

6.1.3.53 I3C_SLAVE_STATUS_EVDET

```
#define I3C_SLAVE_STATUS_EVDET
```

Slave request details: if it was NACKed, ACKed, or sill waiting for START, Bus Available, or Bus-Idle (HJ) conditions

Definition at line 194 of file CDD_I3c_Types.h.

6.1.3.54 I3C_SLAVE_STATUS_EVENT

```
#define I3C_SLAVE_STATUS_EVENT
```

Slave request was sent and ACKed. For more details, please check the Reference Manual

Definition at line 195 of file CDD_I3c_Types.h.

6.1.3.55 I3C_SLAVE_STATUS_CHANDLED

```
#define I3C_SLAVE_STATUS_CHANDLED
```

A Common Command Code (CCC) is being handled by the module

Definition at line 196 of file CDD_I3c_Types.h.

6.1.3.56 I3C_SLAVE_STATUS_HDRMATCHED

```
#define I3C_SLAVE_STATUS_HDRMATCHED
```

An HDR command matched this device's I3C Dynamic Address

Definition at line 197 of file CDD_I3c_Types.h.

6.1.3.57 I3C_SLAVE_STATUS_ERROR_WARNING

```
#define I3C_SLAVE_STATUS_ERROR_WARNING
```

An error or warning has occurred

Definition at line 198 of file CDD_I3c_Types.h.

6.1.3.58 I3C_SLAVE_STATUS_CCC

```
#define I3C_SLAVE_STATUS_CCC
```

A Common Command Code (CCC) has been received, and is not handled by the I3C module

Definition at line 199 of file CDD_I3c_Types.h.

6.1.3.59 I3C_SLAVE_STATUS_DACHG

```
#define I3C_SLAVE_STATUS_DACHG
```

The Slave Dynamic Address (DA) has been assigned, re-assigned, or reset (lost)

Definition at line 200 of file CDD_I3c_Types.h.

6.1.3.60 I3C_SLAVE_STATUS_TXNOTFULL

```
#define I3C_SLAVE_STATUS_TXNOTFULL
```

Tx FIFO not yet full, according to its watermark

Definition at line 201 of file CDD_I3c_Types.h.

6.1.3.61 I3C_SLAVE_STATUS_RXPENDING

```
#define I3C_SLAVE_STATUS_RXPENDING
```

Rx FIFO has receiving data bytes, according to its watermark

Definition at line 202 of file CDD_I3c_Types.h.

6.1.3.62 I3C_SLAVE_STATUS_STOP

```
#define I3C_SLAVE_STATUS_STOP
```

STOP signal was detected on the bus

Definition at line 203 of file CDD_I3c_Types.h.

6.1.3.63 I3C_SLAVE_STATU_MATCHED

```
#define I3C_SLAVE_STATU_MATCHED
```

An incoming header matched this device's I3C Dynamic or I2C Static address (if any)

Definition at line 204 of file CDD_I3c_Types.h.

6.1.3.64 I3C_SLAVE_STATUS_START

```
#define I3C_SLAVE_STATUS_START
```

START or repeated START was seen after the START bit was last cleared

Definition at line 205 of file CDD_I3c_Types.h.

6.1.3.65 I3C_SLAVE_STATUS_STHDR

```
#define I3C_SLAVE_STATUS_STHDR
```

The I3C bus is in HDR-DDR mode. For more details, please check the Reference Manual

Definition at line 206 of file CDD_I3c_Types.h.

6.1.3.66 I3C_SLAVE_STATUS_STDAA

```
#define I3C_SLAVE_STATUS_STDAA
```

The I3C bus is in Enter Dynamic Address Assignment (ENTDAA) mode. For more details, please check the Reference Manual

Definition at line 207 of file CDD_I3c_Types.h.

6.1.3.67 I3C_SLAVE_STATUS_STREQWR

```
#define I3C_SLAVE_STATUS_STREQWR
```

The request in process is SDR write data from the master to this slave. For more details, please check the Reference Manual

Definition at line 208 of file CDD_I3c_Types.h.

6.1.3.68 I3C_SLAVE_STATUS_STREQRD

```
#define I3C_SLAVE_STATUS_STREQRD
```

The request in process is SDR read from this slave, or an IBI is pushed out. For more details, please check the Reference Manual

Definition at line 209 of file CDD_I3c_Types.h.

6.1.3.69 I3C_SLAVE_STATUS_STCCCH

```
#define I3C_SLAVE_STATUS_STCCCH
```

A Common Command Code (CCC) message is being handled automatically

Definition at line 210 of file CDD_I3c_Types.h.

6.1.3.70 I3C_SLAVE_STATUS_STMSG

```
#define I3C_SLAVE_STATUS_STMSG
```

This slave is listening to the bus traffic or responding. For more details, please check the Reference Manual

Definition at line 211 of file CDD_I3c_Types.h.

6.1.3.71 I3C_SLAVE_STATUS_STNOTSTOP

```
#define I3C_SLAVE_STATUS_STNOTSTOP
```

The I3c bus is busy (STOP has not been detected yet)

Definition at line 212 of file CDD_I3c_Types.h.

6.1.3.72 I3C_CCC_ENEC_BROADCAST

```
#define I3C_CCC_ENEC_BROADCAST
```

Common Command Codes. These defines should be used with I3c_SendCommonCommandCode() and I3c_↩ ReadCommonCommandCode(). For more details about each command, please see Mipi Basic Specifications v1.0 document.

Enable events

Definition at line 226 of file CDD_I3c_Types.h.

6.1.3.73 I3C_CCC_DISEC_BROADCAST

```
#define I3C_CCC_DISEC_BROADCAST
```

Disable events

Definition at line 227 of file CDD_I3c_Types.h.

6.1.3.74 I3C_CCC_ENTAS0_BROADCAST

```
#define I3C_CCC_ENTAS0_BROADCAST
```

Enter activity state 0

Definition at line 228 of file CDD_I3c_Types.h.

6.1.3.75 I3C_CCC_ENTAS1_BROADCAST

```
#define I3C_CCC_ENTAS1_BROADCAST
```

Enter activity state 1

Definition at line 229 of file CDD_I3c_Types.h.

6.1.3.76 I3C_CCC_ENTAS2_BROADCAST

```
#define I3C_CCC_ENTAS2_BROADCAST
```

Enter activity state 2

Definition at line 230 of file CDD_I3c_Types.h.

6.1.3.77 I3C_CCC_ENTAS3_BROADCAST

```
#define I3C_CCC_ENTAS3_BROADCAST
```

Enter activity state 3

Definition at line 231 of file CDD_I3c_Types.h.

6.1.3.78 I3C_CCC_RSTDAA_BROADCAST

```
#define I3C_CCC_RSTDAA_BROADCAST
```

Reset Dynamic Address Assignment

Definition at line 232 of file CDD_I3c_Types.h.

6.1.3.79 I3C_CCC_ENTDAA_BROADCAST

```
#define I3C_CCC_ENTDAA_BROADCAST
```

Enter Dynamic Address Assignment

Definition at line 233 of file CDD_I3c_Types.h.

6.1.3.80 I3C_CCC_DEFSLVS_BROADCAST

```
#define I3C_CCC_DEFSLVS_BROADCAST
```

Define list of slaves

Definition at line 234 of file CDD_I3c_Types.h.

6.1.3.81 I3C_CCC_SETMWL_BROADCAST

```
#define I3C_CCC_SETMWL_BROADCAST
```

Set maximum write length

Definition at line 235 of file CDD_I3c_Types.h.

6.1.3.82 I3C_CCC_SETMRL_BROADCAST

```
#define I3C_CCC_SETMRL_BROADCAST
```

Set maximum read length

Definition at line 236 of file CDD_I3c_Types.h.

6.1.3.83 I3C_CCC_ENTTM_BROADCAST

```
#define I3C_CCC_ENTTM_BROADCAST
```

Enter test mode

Definition at line 237 of file CDD_I3c_Types.h.

6.1.3.84 I3C_CCC_ENDXFER_BROADCAST

```
#define I3C_CCC_ENDXFER_BROADCAST
```

Provides the framework for Controllers and Targets to exchange set-up parameters for ending data transfers in supported HDR Modes. For HDR-DDR, it controls Data Transfer Ending Procedure. For more details, please check MIPI I3c Basic Specification document.

Definition at line 240 of file CDD_I3c_Types.h.

6.1.3.85 I3C_CCC_ENTHDR0_BROADCAST

```
#define I3C_CCC_ENTHDR0_BROADCAST
```

Enter HDR-DDR mode

Definition at line 241 of file CDD_I3c_Types.h.

6.1.3.86 I3C_CCC_ENEC_DIRECT

```
#define I3C_CCC_ENEC_DIRECT
```

Enable events

Definition at line 242 of file CDD_I3c_Types.h.

6.1.3.87 I3C_CCC_DISEC_DIRECT

```
#define I3C_CCC_DISEC_DIRECT
```

Disable events

Definition at line 243 of file CDD_I3c_Types.h.

6.1.3.88 I3C_CCC_ENTAS0_DIRECT

```
#define I3C_CCC_ENTAS0_DIRECT
```

Enter activity state 0

Definition at line 244 of file CDD_I3c_Types.h.

6.1.3.89 I3C_CCC_ENTAS1_DIRECT

```
#define I3C_CCC_ENTAS1_DIRECT
```

Enter activity state 1

Definition at line 245 of file CDD_I3c_Types.h.

6.1.3.90 I3C_CCC_ENTAS2_DIRECT

```
#define I3C_CCC_ENTAS2_DIRECT
```

Enter activity state 2

Definition at line 246 of file CDD_I3c_Types.h.

6.1.3.91 I3C_CCC_ENTAS3_DIRECT

```
#define I3C_CCC_ENTAS3_DIRECT
```

Enter activity state 3

Definition at line 247 of file CDD_I3c_Types.h.

6.1.3.92 I3C_CCC_RSTDAA_DIRECT

```
#define I3C_CCC_RSTDAA_DIRECT
```

Reset Dynamic Address Assignment

Definition at line 248 of file CDD_I3c_Types.h.

6.1.3.93 I3C_CCC_SETDASA_DIRECT

```
#define I3C_CCC_SETDASA_DIRECT
```

Set Dynamic Addres from Static Address

Definition at line 249 of file CDD_I3c_Types.h.

6.1.3.94 I3C_CCC_SETNEWDA_DIRECT

```
#define I3C_CCC_SETNEWDA_DIRECT
```

Set New Dynamic Addres

Definition at line 250 of file CDD_I3c_Types.h.

6.1.3.95 I3C_CCC_SETMWL_DIRECT

```
#define I3C_CCC_SETMWL_DIRECT
```

Set maximum write length

Definition at line 251 of file CDD_I3c_Types.h.

6.1.3.96 I3C_CCC_SETMRL_DIRECT

```
#define I3C_CCC_SETMRL_DIRECT
```

Set maximum read length

Definition at line 252 of file CDD_I3c_Types.h.

6.1.3.97 I3C_CCC_GETMWL_DIRECT

```
#define I3C_CCC_GETMWL_DIRECT
```

Get maximum write length

Definition at line 253 of file CDD_I3c_Types.h.

6.1.3.98 I3C_CCC_GETMRL_DIRECT

```
#define I3C_CCC_GETMRL_DIRECT
```

Get maximum read length

Definition at line 254 of file CDD_I3c_Types.h.

6.1.3.99 I3C_CCC_GETPID_DIRECT

```
#define I3C_CCC_GETPID_DIRECT
```

Get Provisional ID

Definition at line 255 of file CDD_I3c_Types.h.

6.1.3.100 I3C_CCC_GETBCR_DIRECT

```
#define I3C_CCC_GETBCR_DIRECT
```

Get Bus Characteristic Register

Definition at line 256 of file CDD_I3c_Types.h.

6.1.3.101 I3C_CCC_GETDCR_DIRECT

```
#define I3C_CCC_GETDCR_DIRECT
```

Get Device Characteristic Register

Definition at line 257 of file CDD_I3c_Types.h.

6.1.3.102 I3C_CCC_GETSTATUS_DIRECT

```
#define I3C_CCC_GETSTATUS_DIRECT
```

Get device status

Definition at line 258 of file CDD_I3c_Types.h.

6.1.3.103 I3C_CCC_GETACCMST_DIRECT

```
#define I3C_CCC_GETACCMST_DIRECT
```

Get accept mastership

Definition at line 259 of file CDD_I3c_Types.h.

6.1.3.104 I3C_CCC_ENDXFER_DIRECT

```
#define I3C_CCC_ENDXFER_DIRECT
```

Provides the framework for Controllers and Targets to exchange set-up parameters for ending data transfers in supported HDR Modes. For HDR-DDR, it controls Data Transfer Ending Procedure. For more details, please check MIPI I3c Basic Specification document.

Definition at line 262 of file CDD_I3c_Types.h.

6.1.3.105 I3C_CCC_SETBRGTGT_DIRECT

```
#define I3C_CCC_SETBRGTGT_DIRECT
```

Set bridge targets

Definition at line 263 of file CDD_I3c_Types.h.

6.1.3.106 I3C_CCC_GETMXDS_DIRECT

```
#define I3C_CCC_GETMXDS_DIRECT
```

Get maximum data speed

Definition at line 264 of file CDD_I3c_Types.h.

6.1.4 Types Reference

6.1.4.1 I3c_PartCoreType

```
typedef uint8 I3c_PartCoreType
```

In-Band Interrupt (IBI) response.

This gives the numeric ID (paritition number) of an I3C Hw Unit.

Definition at line 381 of file CDD_I3c_Types.h.

6.1.4.2 I3c_DataType

```
typedef uint8 I3c_DataType
```

Data Type to be sent or received.

Definition at line 387 of file CDD_I3c_Types.h.

6.1.4.3 I3c_SlaveDeviceType

```
typedef I3c_Ip_SlaveDeviceType I3c_SlaveDeviceType
```

Slave device characteristics structure.

This structure is used to store the characteristics of the I3C slave devices on the bus.

Definition at line 395 of file CDD_I3c_Types.h.

6.1.4.4 I3c_MasterBaudRateType

```
typedef I3c_Ip_MasterBaudRateType I3c_MasterBaudRateType
```

Contains the baud rates options. This structure should be set by the user at runtime.

Definition at line 447 of file CDD_I3c_Types.h.

6.1.5 Enum Reference

6.1.5.1 I3c_MasterSlaveModeType

```
enum I3c_MasterSlaveModeType
```

Definition of the I3C Hw Unit mode.

Enumerator

I3C_MASTER_MODE	The module is an I3C Master
I3C_SLAVE_MODE	The module is an I3C Slave
I3C_MASTER_CAPABLE_MODE	The I3C module is master-capable; however the module is operating as a slave now. When used from the start, the I3C module will start as a slave, but will be prepared to switch to master mode using specific procedures.

Definition at line 274 of file CDD_I3c_Types.h.

6.1.5.2 I3c_DirectionType

```
enum I3c_DirectionType
```

Direction of the transfer: send/receive.

Enumerator

I3C_SEND	Used to send data to a slave.
I3C_RECEIVE	Used to receive data from a slave.

Definition at line 287 of file CDD_I3c_Types.h.

6.1.5.3 I3c_BusType

```
enum I3c_BusType
```

Bus type with EmitStartAddr.

Enumerator

I3C_BUS_TYPE_I3C_SDR	I3C SDR mode
I3C_BUS_TYPE_I2C	I2C mode
I3C_BUS_TYPE_I3C_DDR	I3C HDR-DDR mode

Definition at line 298 of file CDD_I3c_Types.h.

6.1.5.4 I3c_TransferSizeType

enum I3c_TransferSizeType

Indicates size of the transfer message.

Enumerator

I3C_TRANSFER_BYTES	Send/read in bytes
I3C_TRANSFER_HALF_WORDS	Send/read in half-words

Definition at line 309 of file CDD_I3c_Types.h.

6.1.5.5 I3c_StatusType

enum I3c_StatusType

I3C specific error codes.

Enumerator

I3C_STATUS_TIMEOUT	The module has stalled too long in a frame
I3C_STATUS_MAX_SLAVES_EXCEED	Maximum number of devices on the bus has been exceeded
I3C_STATUS_IBI_WON	IBI/MR/HJ won the arbitration on a header address

Definition at line 319 of file CDD_I3c_Types.h.

6.1.5.6 I3c_SlaveRequestType

enum I3c_SlaveRequestType

Indicates the current slave operation mode.

Enumerator

I3C_SLAVE_NORMAL_MODE	Normal mode
I3C_SLAVE_IBI_REQUEST	Start an IBI event
I3C_SLAVE_MASTER_REQUEST	Start a MR or P2P event
I3C_SLAVE_HOT_JOIN_REQUEST	Start a HJ event

Definition at line 350 of file CDD_I3c_Types.h.

6.1.6 Function Reference

6.1.6.1 I3c_Init()

```
void I3c_Init (
            const I3c_ConfigType *const Config )
```

Initialize the I3C module.

This function initializes the I3C module:

- Maps the logical channel to the hardware channel
- Initializes the channel

Parameters

in	*Config*	Pointer to the configuration structure

6.1.6.2 I3c_DeInit()

```
void I3c_DeInit (
            void  )
```

Deinitialize the I3C module.

This function deinitializes the I3C module to the reset values. The driver must be initialized before calling I3c_DeInit().

6.1.6.3 I3c_SyncTransfer()

```
Std_ReturnType I3c_SyncTransfer (
            const uint8 Channel,
            uint8 * DataBuffer,
            const uint32 BufferSize,
            const I3c_RequestType *const Request )
```

Send or receive a blocking message on the I3C bus.

This function is used to send or receive a message, depending on the direction of the transfer. The message is sent or read in a blocking manner. It only returns when the transfer is complete.

Parameters

in	*Channel*	I3C Channel
in	*DataBuffer*	Pointer to the Data buffer
in	*BufferSize*	Buffer size in bytes
in	*Request*	Pointer to the transfer information

Returns

Std_ReturnType

6.1.6.4 I3c_AsyncTransfer()

```
Std_ReturnType I3c_AsyncTransfer (
            const uint8 Channel,
            uint8 * DataBuffer,
            const uint32 BufferSize,
            const I3c_RequestType *const Request )
```

Send or receive a non-blocking message on the I3C bus.

This function is used to send or receive a message, depending on the direction of the transfer. The message is sent or read in a non-blocking manner. It returns immediately. The rest of the transmission is handled by the interrupt service routine.

Parameters

in	*Channel*	I3C Channel
in	*DataBuffer*	Pointer to the Data buffer
in	*BufferSize*	Buffer size in bytes
in	*Request*	Pointer to the transfer information

Returns

Std_ReturnType

6.1.6.5 I3c_GetTransferStatus()

```
I3c_StatusType I3c_GetTransferStatus (
            const uint8 Channel,
            uint32 *const BytesRemaining )
```

Get the status of an I3C channel.

This function is used to get the status of a transfer and check for errors. In addition, if the transfer is still in progress, this functions returns the remaining bytes to be transferred, only if BytesRemaining pointer is different than NULL_PTR.

Parameters

in	*Channel*	I3C Channel
out	*BytesRemaining*	Remaining bytes to be transferred.

Returns

I3c_StatusType

6.1.6.6 I3c_PrepareSlaveBuffer()

```
Std_ReturnType I3c_PrepareSlaveBuffer (
            const uint8 Channel,
            uint8 * DataBuffer,
            const uint8 BufferSize )
```

Prepare the Tx or Rx data buffer for a slave channel.

Prepares the Tx or Rx buffer that will be used to send data or receive data when requested by the master.

Parameters

in	*Channel*	I3C Channel
in	*DataBuffer*	Pointer to the data buffer
in	*BufferSize*	Data buffer size in bytes

Returns

Std_ReturnType

6.1.6.7 I3c_GetError()

```
uint32 I3c_GetError (
            const uint8 Channel )
```

Returns the error register, either for the master or the slave, depending on the configured channel.

This function is used to get the entire error/warning register for the current channel. This may be called from the Error/Warning notification and dedicated defines should be used to filter the errors/warnings which occurred.

Parameters

in	*Channel*	I3C Channel

Returns

Master/Slave error register

6.1.6.8 I3c_PrepareSlaveTransmission()

```
Std_ReturnType I3c_PrepareSlaveTransmission (
            const uint8 Channel )
```

Enables the slave Tx Send event if used in I2C mode.

This function is used to enable Slave TX interrupt. This is used to fill the Slave TX FIFO before master request. Note that before calling this function, I3c_PrepareSlaveBuffer() should be called.

Parameters

in	*Channel*	I3C Channel

6.1.6.9 I3c_MasterSetBaudRate()

```
Std_ReturnType I3c_MasterSetBaudRate (
            const uint8 Channel,
            const uint32 FunctionalClock,
            const I3c_MasterBaudRateType * BaudRates,
            const I3c_BusType BusType )
```

Set the baud rate for all subsequent I3C communications.

This function is used to set the baud rate for all subsequent I3C communications. Using this function, the baud rates will be set as close as possible to the target ones. After calling this function, I3c_MasterGetBaudRate() shall be called to check which baud rates were actually set.

Parameters

in	*Channel*	I3C Channel
in	*FunctionalClock*	Value of the Functional Clock (Hz)
in	*BaudRates*	Pointer to the Baud rates structure: PP, OD, I2C (Hz)
in	*BusType*	Type of the bus: I2C, SDR, DDR

6.1.6.10 I3c_MasterGetBaudRate()

```
Std_ReturnType I3c_MasterGetBaudRate (
            const uint8 Channel,
            const uint32 FunctionalClock,
            I3c_MasterBaudRateType * BaudRates )
```

Get the baud rates of the I3C module.

This function is used to get the baud rates of the I3C module.

Parameters

in	*Channel*	I3C Channel
in	*FunctionalClock*	Value of the Functional Clock (Hz)
out	*BaudRates*	Pointer to the Baud rates structure: PP, OD, I2C (Hz)

6.1.6.11 I3c_GetStatus()

```
uint32 I3c_GetStatus (
            const uint8 Channel )
```

Returns the error register, either for the master or the slave, depending on the configured channel.

This function is used to get the entire status register for the current channel. This may be called from the provided notification and dedicated defines should be used to check which specific status is set.

Parameters

in	*Channel*	I3C Channel

Returns

Master/Slave status register

6.2 I3c_ip

6.2.1 Detailed Description

Data Structures

- struct I3c_Ip_TransferConfigType

  Contains the transfer options. This structure must be configured at runtime, before calling Send/Receive API. More...
- struct I3c_Ip_MasterBaudRateType

  Contains the baud rates options. This structure should be set by the user at runtime. More...
- struct I3c_Ip_MasterStateType

  Master internal context structure. More...
- struct I3c_Ip_MasterConfigType

  Master configuration structure. More...
- struct I3c_Ip_SlaveStateType

  Slave internal context structure. More...
- struct I3c_Ip_SlaveConfigType

  Slave configuration structure. More...
- struct I3c_Ip_SlaveDeviceType

  Slave device characteristics structure. More...

Macros

- #define I3C_IP_BROADCAST_ADDRESS

  I3c broadcast address.
- #define I3C_IP_MASTER_ERROR_NACK

  I3C master error flags.
- #define I3C_IP_MASTER_ERROR_WRITE_ABORT
- #define I3C_IP_MASTER_ERROR_TERM
- #define I3C_IP_MASTER_ERROR_PARITY
- #define I3C_IP_MASTER_ERROR_CRC
- #define I3C_IP_MASTER_ERROR_OREAD
- #define I3C_IP_MASTER_ERROR_OWRITE
- #define I3C_IP_MASTER_ERROR_MSG
- #define I3C_IP_MASTER_ERROR_INVALID_REQUEST
- #define I3C_IP_MASTER_ERROR_TIMEOUT
- #define I3C_IP_SLAVE_ERROR_RX_OVERRUN

  I3C slave error flags.
- #define I3C_IP_SLAVE_ERROR_TX_UNDERRUN
- #define I3C_IP_SLAVE_ERROR_TX_UNDERRUN_NACK
- #define I3C_IP_SLAVE_ERROR_TERM
- #define I3C_IP_SLAVE_ERROR_INVALID_START
- #define I3C_IP_SLAVE_ERROR_SDR_PARITY
- #define I3C_IP_SLAVE_ERROR_HDR_PARITY_ERROR
- #define I3C_IP_SLAVE_ERROR_HDR_CRC
- #define I3C_IP_SLAVE_ERROR_S0S1

- #define I3C_IP_SLAVE_ERROR_OREAD
- #define I3C_IP_SLAVE_ERROR_OWRITE
- #define I3C_IP_MASTER_STATUS_IBIADDR

  I3C master status flags.
- #define I3C_IP_MASTER_STATUS_NOWMASTER
- #define I3C_IP_MASTER_STATUS_ERROR_WARNING
- #define I3C_IP_MASTER_STATUS_IBIWON
- #define I3C_IP_MASTER_STATUS_TXNOTFULL
- #define I3C_IP_MASTER_STATUS_RXPENDING
- #define I3C_IP_MASTER_STATUS_COMPLETE
- #define I3C_IP_MASTER_STATUS_CTRLDONE
- #define I3C_IP_MASTER_STATUS_SLVSTART
- #define I3C_IP_MASTER_STATUS_IBITYPE
- #define I3C_IP_MASTER_STATUS_NACKED
- #define I3C_IP_MASTER_STATUS_BETWEEN
- #define I3C_IP_MASTER_STATUS_STATE
- #define I3C_IP_SLAVE_STATUS_TIMECTRL

  I3C slave status flags.
- #define I3C_IP_SLAVE_STATUS_ACTSTATE
- #define I3C_IP_SLAVE_STATUS_HJDIS
- #define I3C_IP_SLAVE_STATUS_MRDIS
- #define I3C_IP_SLAVE_STATUS_IBIDIS
- #define I3C_IP_SLAVE_STATUS_EVDET
- #define I3C_IP_SLAVE_STATUS_EVENT
- #define I3C_IP_SLAVE_STATUS_CHANDLED
- #define I3C_IP_SLAVE_STATUS_HDRMATCHED
- #define I3C_IP_SLAVE_STATUS_ERROR_WARNING
- #define I3C_IP_SLAVE_STATUS_CCC
- #define I3C_IP_SLAVE_STATUS_DACHG
- #define I3C_IP_SLAVE_STATUS_TXNOTFULL
- #define I3C_IP_SLAVE_STATUS_RXPENDING
- #define I3C_IP_SLAVE_STATUS_STOP
- #define I3C_IP_SLAVE_STATU_MATCHED
- #define I3C_IP_SLAVE_STATUS_START
- #define I3C_IP_SLAVE_STATUS_STHDR
- #define I3C_IP_SLAVE_STATUS_STDAA
- #define I3C_IP_SLAVE_STATUS_STREQWR
- #define I3C_IP_SLAVE_STATUS_STREQRD
- #define I3C_IP_SLAVE_STATUS_STCCCH
- #define I3C_IP_SLAVE_STATUS_STMSG
- #define I3C_IP_SLAVE_STATUS_STNOTSTOP
- #define I3C_IP_MASTER_MODE_IDLE

  Indicates the current master operation mode.
- #define I3C_IP_MASTER_MODE_SLAVE_REQUEST
- #define I3C_IP_MASTER_MODE_MSG_SDR
- #define I3C_IP_MASTER_MODE_NORMAL_ACTIVE_SDR
- #define I3C_IP_MASTER_MODE_MSG_DDR
- #define I3C_IP_MASTER_MODE_DAA
- #define I3C_IP_MASTER_MODE_IBI_ACK_NACK
- #define I3C_IP_MASTER_MODE_IBI_RECEIVE

- #define I3C_IP_CCC_ENEC_BROADCAST

  Common Command Codes. These defines should be used with I3c_SendCommonCommandCode() and I3c_Read↩ CommonCommandCode(). For more details about each command, please see Mipi Basic Specifications v1.0 document.
- #define I3C_IP_CCC_DISEC_BROADCAST
- #define I3C_IP_CCC_ENTAS0_BROADCAST
- #define I3C_IP_CCC_ENTAS1_BROADCAST
- #define I3C_IP_CCC_ENTAS2_BROADCAST
- #define I3C_IP_CCC_ENTAS3_BROADCAST
- #define I3C_IP_CCC_RSTDAA_BROADCAST
- #define I3C_IP_CCC_ENTDAA_BROADCAST
- #define I3C_IP_CCC_DEFSLVS_BROADCAST
- #define I3C_IP_CCC_SETMWL_BROADCAST
- #define I3C_IP_CCC_SETMRL_BROADCAST
- #define I3C_IP_CCC_ENTTM_BROADCAST
- #define I3C_IP_CCC_ENDXFER_BROADCAST
- #define I3C_IP_CCC_ENTHDR0_BROADCAST
- #define I3C_IP_CCC_ENEC_DIRECT
- #define I3C_IP_CCC_DISEC_DIRECT
- #define I3C_IP_CCC_ENTAS0_DIRECT
- #define I3C_IP_CCC_ENTAS1_DIRECT
- #define I3C_IP_CCC_ENTAS2_DIRECT
- #define I3C_IP_CCC_ENTAS3_DIRECT
- #define I3C_IP_CCC_RSTDAA_DIRECT
- #define I3C_IP_CCC_SETDASA_DIRECT
- #define I3C_IP_CCC_SETNEWDA_DIRECT
- #define I3C_IP_CCC_SETMWL_DIRECT
- #define I3C_IP_CCC_SETMRL_DIRECT
- #define I3C_IP_CCC_GETMWL_DIRECT
- #define I3C_IP_CCC_GETMRL_DIRECT
- #define I3C_IP_CCC_GETPID_DIRECT
- #define I3C_IP_CCC_GETBCR_DIRECT
- #define I3C_IP_CCC_GETDCR_DIRECT
- #define I3C_IP_CCC_GETSTATUS_DIRECT
- #define I3C_IP_CCC_GETACCMST_DIRECT
- #define I3C_IP_CCC_ENDXFER_DIRECT
- #define I3C_IP_CCC_SETBRGTGT_DIRECT
- #define I3C_IP_CCC_GETMXDS_DIRECT

Types Reference

- typedef void(* I3c_Ip_MasterCallbackType) (I3c_Ip_MasterEventType Event)

  Callback for all peripherals which supports I3C features for master mode.
- typedef void(* I3c_Ip_SlaveCallbackType) (I3c_Ip_SlaveEventType Event)

  Callback for all peripherals which supports I3C features for slave mode.

Enum Reference

- enum I3c_Ip_MasterRequestType

  Emits the requested operation when doing in pieces vs. by message.
- enum I3c_Ip_MasterEventType

  Events which can trigger I3C master callback.
- enum I3c_Ip_MasterEnableType

  Indicates if the master is enabled and what states are allowed for it (the master).
- enum I3c_Ip_MasterHighKeepType

  Indicates how High-Keeper is to be supported.
- enum I3c_Ip_BusType

  Bus type with EmitStartAddr.
- enum I3c_Ip_DirectionType

  Direction: send/receive.
- enum I3c_Ip_FifoTriggerLevelType

  Watermark of Tx/RX INT/DMA trigger level.
- enum I3c_Ip_StatusType

  I3C specific error codes.
- enum I3c_Ip_TransferSizeType

  Indicates size of the transfer message.
- enum I3c_Ip_TransferType

  Type of I3C transfer (based on interrupts or DMA)
- enum I3c_Ip_SlaveRequestType

  Indicates the current slave operation mode.
- enum I3c_Ip_SlaveEventType

  Events which can trigger I3C slave callback.

Function Reference

- void I3c_Ip_MasterInit (const uint8 Instance, const I3c_Ip_MasterConfigType *const Config)

  Initialize the I3C master module.
- void I3c_Ip_MasterDeInit (const uint8 Instance)

  Deinitialize the I3C master module.
- I3c_Ip_StatusType I3c_Ip_MasterGetTransferStatus (const uint8 Instance, uint32 *const BytesRemaining)

  Return the current status of the I3C master transfer.
- I3c_Ip_StatusType I3c_Ip_MasterSendBlocking (const uint8 Instance, const uint8 *TxBuffer, const uint32 TxBufferSize, const I3c_Ip_TransferConfigType *const TransferOption)

  Perform a blocking send transaction on the I3C bus.
- I3c_Ip_StatusType I3c_Ip_MasterReceiveBlocking (const uint8 Instance, uint8 *RxBuffer, const uint32 Rx↩ BufferSize, const I3c_Ip_TransferConfigType *const TransferOption)

  Perform a blocking receive transaction on the I3C bus.
- I3c_Ip_StatusType I3c_Ip_MasterSend (const uint8 Instance, const uint8 *TxBuffer, const uint32 TxBuffer↩ Size, const I3c_Ip_TransferConfigType *const TransferOption)

  Perform a non-blocking send transaction on the I3C bus.
- I3c_Ip_StatusType I3c_Ip_MasterReceive (const uint8 Instance, uint8 *RxBuffer, const uint32 RxBufferSize, const I3c_Ip_TransferConfigType *const TransferOption)

Perform a non-blocking receive transaction on the I3C bus.

- void I3c_Ip_SlaveInit (const uint8 Instance, const I3c_Ip_SlaveConfigType *const Config)

  Initialize the I3C slave module.
- void I3c_Ip_SlaveDeInit (const uint8 Instance)

  Deinitialize the I3C slave module.
- I3c_Ip_StatusType I3c_Ip_SlaveGetTransferStatus (const uint8 Instance, uint32 *const BytesRemaining)

  Return the current status of the I3C slave transfer.
- void I3c_Ip_MasterWriteOwnDynamicAddress (const uint8 Instance, const uint8 Address, const boolean Valid)

  The main master assigns its own Dynamic Address before handing off the mastership.
- void I3c_Ip_SlaveSetMapAddress (const uint8 Instance, const boolean AddressType, const uint16 Address, const boolean Nack, const boolean Enable)

  Configure a slave which was the main master and just handed off the mastership.
- uint32 I3c_Ip_MasterGetError (const uint8 Instance)

  Returns the master error register.
- uint32 I3c_Ip_SlaveGetError (const uint8 Instance)

  Returns the slave error register.
- void I3c_Ip_SlaveSetBuffer (const uint8 Instance, uint8 *DataBuffer, const uint8 BufferSize)

  Provide a buffer for the slave's data transfer.
- void I3c_Ip_SlavePrepareTxFifo (const uint8 Instance)

  Enables the slave Tx Send event if used in I2C mode.
- I3c_Ip_StatusType I3c_Ip_MasterSetBaudRate (const uint8 Instance, const uint32 FunctionalClock, const I3c_Ip_MasterBaudRateType *BaudRates, const I3c_Ip_BusType BusType)

  Set the baud rate for all subsequent I3C communications.
- void I3c_Ip_MasterGetBaudRate (const uint8 Instance, const uint32 FunctionalClock, I3c_Ip_MasterBaudRateType *BaudRates)

  Get the baud rates of the I3C module.
- uint32 I3c_Ip_MasterGetStatus (const uint8 Instance)

  Returns the master status register.
- uint32 I3c_Ip_SlaveGetStatus (const uint8 Instance)

  Returns the master status register.

6.2.2 Data Structure Documentation

6.2.2.1 struct I3c_Ip_TransferConfigType

Contains the transfer options. This structure must be configured at runtime, before calling Send/Receive API.

Definition at line 485 of file I3c_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	SlaveAddress	Slave Device Address
boolean	SendStop	Specifies if either a STOP or repeated START signal shall be emitted
I3c_Ip_DirectionType	Direction	Specifies the direction of the transfer, send or receive
I3c_Ip_TransferSizeType	TransferSize	Specifies if the transfer is in half words or in bytes
I3c_Ip_BusType	BusType	Bus Type: SDR, DDR, I2C

6.2.2.2 struct I3c_Ip_MasterBaudRateType

Contains the baud rates options. This structure should be set by the user at runtime.

Definition at line 499 of file I3c_Ip_Types.h.

Data Fields

Type	Name	Description
uint32	I2cBaudRate	Desired I2C baud rate in Hz
uint32	OpenDrainBaudRate	Desired Open-Drain baud rate in Hz
uint32	PushPullBaudRate	Desired push-pull baud rate in Hz

6.2.2.3 struct I3c_Ip_MasterStateType

Master internal context structure.

This structure is used by the master-mode driver for its internal logic.

Definition at line 529 of file I3c_Ip_Types.h.

Data Fields

Type	Name	Description
uint32	BufferSize	Tx/Rx buffer size
const uint8 *	TxDataBuffer	Pointer to Tx data buffer
uint8 *	RxDataBuffer	Pointer to Rx data buffer
volatile I3c_Ip_StatusType	Status	Status of the last driver operation
I3c_Ip_TransferConfigType	TransferOption	Transfer options
I3c_Ip_TransferType	TransferType	Type of the transfer: DMA/interrupts
I3c_Ip_MasterCallbackType	MasterCallback	Master callback function

6.2.2.4 struct I3c_Ip_MasterConfigType

Master configuration structure.

This structure is used to provide configuration parameters for the I3C master.

Definition at line 561 of file I3c_Ip_Types.h.

Data Fields

Type	Name	Description
I3c_Ip_MasterEnableType	MasterEnable	Enable master mode

Data Fields

Type	Name	Description
boolean	DisableTimeout	Disable timeout to prevent application errors (MERRWARN)
I3c_Ip_MasterHighKeepType	MasterHighKeeper	High keeper mode setting
uint8	I2cBaud	I2C baud in cycles
uint8	OpenDrainBaud	I3C open-drain baud rate in cycles
uint8	PushPullBaud	I3C push-pull baud rate in cycles
uint8	PushPullLow	Adder for push-pull low, to create a duty-cycle with a longer low period
boolean	OpenDrainHighPP	Optional field that allows for the I3C open drain to be long low and short high
boolean	OpenDrainStop	If true, then STOP will be emitted at open-drain speeds even for I3C messages
uint8	Skew	Number of FCLK counts for an SDA change after SCL for I3C push-pull
boolean	EnableStartOptimization	If this is true, then all I3C Dynamic Addresses must be less than 0x40. This improves the speed for sending the START header
boolean	NoMandatoryByte	If true, then the ADDRx from In-Band Interrupt Rules register refer to slaves without mandatory byte
I3c_Ip_MasterStateType *	MasterState	Pointer to the master state structure

6.2.2.5 struct I3c_Ip_SlaveStateType

Slave internal context structure.

This structure is used by the slave-mode driver for its internal logic.

Definition at line 590 of file I3c_Ip_Types.h.

Data Fields

Type	Name	Description
uint32	BufferSize	Tx/Rx buffer size
uint8 *	DataBuffer	Pointer to data buffer
volatile I3c_Ip_StatusType	Status	Slave status
uint8	RepeatedStarts	Number of Repeated STARTs on the I3c bus, before a STOP was detected. When STOP signal is detected, this counter is reseted.
I3c_Ip_DirectionType	Direction	Specifies the direction of the transfer, send or receive
I3c_Ip_TransferSizeType	TransferSize	Transfer size: bytes or half words
I3c_Ip_TransferType	TransferType	Type of the transfer: DMA/interrupts
I3c_Ip_SlaveCallbackType	SlaveCallback	Slave callback function

6.2.2.6 struct I3c_Ip_SlaveConfigType

Slave configuration structure.

This structure is used to provide configuration parameters for the I3C slave.

Definition at line 617 of file I3c_Ip_Types.h.

Data Fields

Type	Name	Description
boolean	SlaveEnable	If true, then a slave can operate on the I2C or I3C bus. Otherwise, ignore I2C or I3C bus
boolean	Nack	If true, then the slave will NACK all requests to it, except a CCC broadcast
boolean	MatchStartStop	Whether to assert START/STOP status only if the slave is addressed
boolean	IgnoreS0S1Error	Whether to ignore S0/S1 error in SDR mode
boolean	HighDataRateSupport	If true, the slave allow HDR-DDR messaging, if available
boolean	OfflineSupport	Whether to wait 60 us of bus quiet or HDR Exit Pattern to ensure bus is not in HDR mode and safely track the message
uint8	BusAvailableMatch	Provides the count of the slow clock to count out 1 us (or more), to allow an In-Band Interrupt (IBI) to drive SDA low
uint8	StaticAddress	Slave's Static Address
uint8	BCR	Bus Characteristic Register
uint8	DCR	Device Characteristic Register
uint16	VendorId	Slave Vendor ID
uint32	PartNumber	Slave ID Part Number
uint8	VendorInformation	Vendor Information
uint8	ActivityState	Slave Activity State
uint8	PendingInt	Pending Interrupt Number
uint8	MapIndex	Index of Dynamic Address that the In-Band Interrupt is for
uint8	ClockFrequency	Slave control clock frequency
uint8	ClockAccuracy	Slave control clock accuracy
I3c_Ip_SlaveStateType *	SlaveState	Pointer to the slave state structure

6.2.2.7 struct I3c_Ip_SlaveDeviceType

Slave device characteristics structure.

This structure is used to store the characteristics of the I3C slave devices on the bus.

Definition at line 649 of file I3c_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	BCR	Bus Characteristic Register
uint8	DCR	Device Characteristic Register
uint16	VendorId	Slave Vendor ID
uint32	PartNumber	Slave ID Part Number

6.2.3 Macro Definition Documentation

6.2.3.1 I3C_IP_BROADCAST_ADDRESS

```
#define I3C_IP_BROADCAST_ADDRESS
```

I3c broadcast address.

All I3c slave devices shall match this address. In addition, no I2c slave device shall match this address because it is reserved and unused in I2c.

Definition at line 96 of file I3c_Ip_Types.h.

6.2.3.2 I3C_IP_MASTER_ERROR_NACK

```
#define I3C_IP_MASTER_ERROR_NACK
```

I3C master error flags.

These can be used with I3c_Ip_MasterGetError() function to filter the active errors. Use these masks to check which specific errors/warnings occurred. Slave(s) NACKed the last address

Definition at line 105 of file I3c_Ip_Types.h.

6.2.3.3 I3C_IP_MASTER_ERROR_WRITE_ABORT

```
#define I3C_IP_MASTER_ERROR_WRITE_ABORT
```

Slave NACKed the write data

Definition at line 106 of file I3c_Ip_Types.h.

6.2.3.4 I3C_IP_MASTER_ERROR_TERM

```
#define I3C_IP_MASTER_ERROR_TERM
```

Master terminated a slave read

Definition at line 107 of file I3c_Ip_Types.h.

6.2.3.5 I3C_IP_MASTER_ERROR_PARITY

```
#define I3C_IP_MASTER_ERROR_PARITY
```

Parity error from a DDR read

Definition at line 108 of file I3c_Ip_Types.h.

6.2.3.6 I3C_IP_MASTER_ERROR_CRC

```
#define I3C_IP_MASTER_ERROR_CRC
```

CRC error from a DDR read

Definition at line 109 of file I3c_Ip_Types.h.

6.2.3.7 I3C_IP_MASTER_ERROR_OREAD

```
#define I3C_IP_MASTER_ERROR_OREAD
```

Read from MRDATAB register when FIFO empty

Definition at line 110 of file I3c_Ip_Types.h.

6.2.3.8 I3C_IP_MASTER_ERROR_OWRITE

```
#define I3C_IP_MASTER_ERROR_OWRITE
```

Write to MWDATAB register when FIFO full

Definition at line 111 of file I3c_Ip_Types.h.

6.2.3.9 I3C_IP_MASTER_ERROR_MSG

```
#define I3C_IP_MASTER_ERROR_MSG
```

Read/write SDR/DDR mismatch

Definition at line 112 of file I3c_Ip_Types.h.

6.2.3.10 I3C_IP_MASTER_ERROR_INVALID_REQUEST

```
#define I3C_IP_MASTER_ERROR_INVALID_REQUEST
```

Invalid use of a request

Definition at line 113 of file I3c_Ip_Types.h.

6.2.3.11 I3C_IP_MASTER_ERROR_TIMEOUT

```
#define I3C_IP_MASTER_ERROR_TIMEOUT
```

The module has stalled too long in a frame

Definition at line 114 of file I3c_Ip_Types.h.

6.2.3.12 I3C_IP_SLAVE_ERROR_RX_OVERRUN

```
#define I3C_IP_SLAVE_ERROR_RX_OVERRUN
```

I3C slave error flags.

These can be used with I3c_Ip_SlaveGetError() function to filter the active errors. Use these masks to check which specific errors/warnings occurred. Slave internal RX buffer/FIFO overrun

Definition at line 123 of file I3c_Ip_Types.h.

6.2.3.13 I3C_IP_SLAVE_ERROR_TX_UNDERRUN

```
#define I3C_IP_SLAVE_ERROR_TX_UNDERRUN
```

Slave internal TX buffer/FIFO underrun

Definition at line 124 of file I3c_Ip_Types.h.

6.2.3.14 I3C_IP_SLAVE_ERROR_TX_UNDERRUN_NACK

```
#define I3C_IP_SLAVE_ERROR_TX_UNDERRUN_NACK
```

The internal Tx buffer/FIFO was underrun in the read header and so the module NACKed the header

Definition at line 125 of file I3c_Ip_Types.h.

6.2.3.15 I3C_IP_SLAVE_ERROR_TERM

```
#define I3C_IP_SLAVE_ERROR_TERM
```

Master terminated a read from slave

Definition at line 126 of file I3c_Ip_Types.h.

6.2.3.16 I3C_IP_SLAVE_ERROR_INVALID_START

```
#define I3C_IP_SLAVE_ERROR_INVALID_START
```

Invalid start with SCL falling while SDA = 1 is in a STOP condition

Definition at line 127 of file I3c_Ip_Types.h.

6.2.3.17 I3C_IP_SLAVE_ERROR_SDR_PARITY

```
#define I3C_IP_SLAVE_ERROR_SDR_PARITY
```

SDR parity error on message from master

Definition at line 128 of file I3c_Ip_Types.h.

6.2.3.18 I3C_IP_SLAVE_ERROR_HDR_PARITY_ERROR

```
#define I3C_IP_SLAVE_ERROR_HDR_PARITY_ERROR
```

HDR parity error on message from master

Definition at line 129 of file I3c_Ip_Types.h.

6.2.3.19 I3C_IP_SLAVE_ERROR_HDR_CRC

```
#define I3C_IP_SLAVE_ERROR_HDR_CRC
```

HDR-DDR CRC error on a message from master

Definition at line 130 of file I3c_Ip_Types.h.

6.2.3.20 I3C_IP_SLAVE_ERROR_S0S1

```
#define I3C_IP_SLAVE_ERROR_S0S1
```

S0 or S1 error

Definition at line 131 of file I3c_Ip_Types.h.

6.2.3.21 I3C_IP_SLAVE_ERROR_OREAD

```
#define I3C_IP_SLAVE_ERROR_OREAD
```

The SRDATAB register was read for more bytes than were available

Definition at line 132 of file I3c_Ip_Types.h.

6.2.3.22 I3C_IP_SLAVE_ERROR_OWRITE

```
#define I3C_IP_SLAVE_ERROR_OWRITE
```

The SWDATA register was written when FULL

Definition at line 133 of file I3c_Ip_Types.h.

6.2.3.23 I3C_IP_MASTER_STATUS_IBIADDR

```
#define I3C_IP_MASTER_STATUS_IBIADDR
```

I3C master status flags.

These can be used with I3c_Ip_MasterGetStatus() function to filter the active status flags. Use these masks to check which specific status is set. The address of the Slave Request (IBI, MR, HJ)

Definition at line 142 of file I3c_Ip_Types.h.

6.2.3.24 I3C_IP_MASTER_STATUS_NOWMASTER

```
#define I3C_IP_MASTER_STATUS_NOWMASTER
```

Now this module is a master, but was previously a slave and mastership was handed

Definition at line 143 of file I3c_Ip_Types.h.

6.2.3.25 I3C_IP_MASTER_STATUS_ERROR_WARNING

```
#define I3C_IP_MASTER_STATUS_ERROR_WARNING
```

An error or warning occurred

Definition at line 144 of file I3c_Ip_Types.h.

6.2.3.26 I3C_IP_MASTER_STATUS_IBIWON

```
#define I3C_IP_MASTER_STATUS_IBIWON
```

A slave request won the arbitration on a header address, regardless of whether it was ACKed/NACKed

Definition at line 145 of file I3c_Ip_Types.h.

6.2.3.27 I3C_IP_MASTER_STATUS_TXNOTFULL

```
#define I3C_IP_MASTER_STATUS_TXNOTFULL
```

Tx FIFO not yet full, according to its watermark

Definition at line 146 of file I3c_Ip_Types.h.

6.2.3.28 I3C_IP_MASTER_STATUS_RXPENDING

```
#define I3C_IP_MASTER_STATUS_RXPENDING
```

Rx FIFO has receiving data bytes, according to its watermark

Definition at line 147 of file I3c_Ip_Types.h.

6.2.3.29 I3C_IP_MASTER_STATUS_COMPLETE

```
#define I3C_IP_MASTER_STATUS_COMPLETE
```

A message has completed. For more details, please check the Reference Manual

Definition at line 148 of file I3c_Ip_Types.h.

6.2.3.30 I3C_IP_MASTER_STATUS_CTRLDONE

```
#define I3C_IP_MASTER_STATUS_CTRLDONE
```

A Master Control request was completed. For more details, please check the Reference Manual

Definition at line 149 of file I3c_Ip_Types.h.

6.2.3.31 I3C_IP_MASTER_STATUS_SLVSTART

```
#define I3C_IP_MASTER_STATUS_SLVSTART
```

A slave is/was requesting a START by pulling the SDA low

Definition at line 150 of file I3c_Ip_Types.h.

6.2.3.32 I3C_IP_MASTER_STATUS_IBITYPE

```
#define I3C_IP_MASTER_STATUS_IBITYPE
```

Slave request type (IBI, MR, HJ)

Definition at line 151 of file I3c_Ip_Types.h.

6.2.3.33 I3C_IP_MASTER_STATUS_NACKED

```
#define I3C_IP_MASTER_STATUS_NACKED
```

The last address header was NACKed by the addressed slave

Definition at line 152 of file I3c_Ip_Types.h.

6.2.3.34 I3C_IP_MASTER_STATUS_BETWEEN

```
#define I3C_IP_MASTER_STATUS_BETWEEN
```

Between messages or Dynamic Address Assignment. For more details, please check the Reference Manual

Definition at line 153 of file I3c_Ip_Types.h.

6.2.3.35 I3C_IP_MASTER_STATUS_STATE

```
#define I3C_IP_MASTER_STATUS_STATE
```

Indicates the current master state

Definition at line 154 of file I3c_Ip_Types.h.

6.2.3.36 I3C_IP_SLAVE_STATUS_TIMECTRL

```
#define I3C_IP_SLAVE_STATUS_TIMECTRL
```

I3C slave status flags.

These can be used with I3c_Ip_SlaveGetStatus() function to filter the active status flags. Use these masks to check which specific status is set. Indicates if time control is currently enabled

Definition at line 169 of file I3c_Ip_Types.h.

6.2.3.37 I3C_IP_SLAVE_STATUS_ACTSTATE

```
#define I3C_IP_SLAVE_STATUS_ACTSTATE
```

Activity state from Common Command Codes (CCC)

Definition at line 170 of file I3c_Ip_Types.h.

6.2.3.38 I3C_IP_SLAVE_STATUS_HJDIS

```
#define I3C_IP_SLAVE_STATUS_HJDIS
```

Hot-Join is disabled at this time. While Hot-Join is disabled, MCTRL requests will be held off

Definition at line 171 of file I3c_Ip_Types.h.

6.2.3.39 I3C_IP_SLAVE_STATUS_MRDIS

```
#define I3C_IP_SLAVE_STATUS_MRDIS
```

Master Requests are disabled at this time. While Master Requests are disabled, MCTRL requests will be held off

Definition at line 172 of file I3c_Ip_Types.h.

6.2.3.40 I3C_IP_SLAVE_STATUS_IBIDIS

```
#define I3C_IP_SLAVE_STATUS_IBIDIS
```

In-Band interrupts are disabled at this time. While Master Requests are disabled, MCTRL requests will be held off

Definition at line 173 of file I3c_Ip_Types.h.

6.2.3.41 I3C_IP_SLAVE_STATUS_EVDET

```
#define I3C_IP_SLAVE_STATUS_EVDET
```

Slave request details: if it was NACKed, ACKed, or sill waiting for START, Bus Available, or Bus-Idle (HJ) conditions

Definition at line 174 of file I3c_Ip_Types.h.

6.2.3.42 I3C_IP_SLAVE_STATUS_EVENT

```
#define I3C_IP_SLAVE_STATUS_EVENT
```

Slave request was sent and ACKed. For more details, please check the Reference Manual

Definition at line 175 of file I3c_Ip_Types.h.

6.2.3.43 I3C_IP_SLAVE_STATUS_CHANDLED

```
#define I3C_IP_SLAVE_STATUS_CHANDLED
```

A Common Command Code (CCC) is being handled by the module

Definition at line 176 of file I3c_Ip_Types.h.

6.2.3.44 I3C_IP_SLAVE_STATUS_HDRMATCHED

```
#define I3C_IP_SLAVE_STATUS_HDRMATCHED
```

An HDR command matched this device's I3C Dynamic Address

Definition at line 177 of file I3c_Ip_Types.h.

6.2.3.45 I3C_IP_SLAVE_STATUS_ERROR_WARNING

```
#define I3C_IP_SLAVE_STATUS_ERROR_WARNING
```

An error or warning has occurred

Definition at line 178 of file I3c_Ip_Types.h.

6.2.3.46 I3C_IP_SLAVE_STATUS_CCC

```
#define I3C_IP_SLAVE_STATUS_CCC
```

A Common Command Code (CCC) has been received, and is not handled by the I3C module

Definition at line 179 of file I3c_Ip_Types.h.

6.2.3.47 I3C_IP_SLAVE_STATUS_DACHG

```
#define I3C_IP_SLAVE_STATUS_DACHG
```

The Slave Dynamic Address (DA) has been assigned, re-assigned, or reset (lost)

Definition at line 180 of file I3c_Ip_Types.h.

6.2.3.48 I3C_IP_SLAVE_STATUS_TXNOTFULL

```
#define I3C_IP_SLAVE_STATUS_TXNOTFULL
```

Tx FIFO not yet full, according to its watermark

Definition at line 181 of file I3c_Ip_Types.h.

6.2.3.49 I3C_IP_SLAVE_STATUS_RXPENDING

```
#define I3C_IP_SLAVE_STATUS_RXPENDING
```

Rx FIFO has receiving data bytes, according to its watermark

Definition at line 182 of file I3c_Ip_Types.h.

6.2.3.50 I3C_IP_SLAVE_STATUS_STOP

```
#define I3C_IP_SLAVE_STATUS_STOP
```

STOP signal was detected on the bus

Definition at line 183 of file I3c_Ip_Types.h.

6.2.3.51 I3C_IP_SLAVE_STATU_MATCHED

```
#define I3C_IP_SLAVE_STATU_MATCHED
```

An incoming header matched this device's I3C Dynamic or I2C Static address (if any)

Definition at line 184 of file I3c_Ip_Types.h.

6.2.3.52 I3C_IP_SLAVE_STATUS_START

```
#define I3C_IP_SLAVE_STATUS_START
```

START or repeated START was seen after the START bit was last cleared

Definition at line 185 of file I3c_Ip_Types.h.

6.2.3.53 I3C_IP_SLAVE_STATUS_STHDR

```
#define I3C_IP_SLAVE_STATUS_STHDR
```

The I3C bus is in HDR-DDR mode. For more details, please check the Reference Manual

Definition at line 186 of file I3c_Ip_Types.h.

6.2.3.54 I3C_IP_SLAVE_STATUS_STDAA

```
#define I3C_IP_SLAVE_STATUS_STDAA
```

The I3C bus is in Enter Dynamic Address Assignment (ENTDAA) mode. For more details, please check the Reference Manual

Definition at line 187 of file I3c_Ip_Types.h.

6.2.3.55 I3C_IP_SLAVE_STATUS_STREQWR

```
#define I3C_IP_SLAVE_STATUS_STREQWR
```

The request in process is SDR write data from the master to this slave. For more details, please check the Reference Manual

Definition at line 188 of file I3c_Ip_Types.h.

6.2.3.56 I3C_IP_SLAVE_STATUS_STREQRD

```
#define I3C_IP_SLAVE_STATUS_STREQRD
```

The request in process is SDR read from this slave, or an IBI is pushed out. For more details, please check the Reference Manual

Definition at line 189 of file I3c_Ip_Types.h.

6.2.3.57 I3C_IP_SLAVE_STATUS_STCCCH

```
#define I3C_IP_SLAVE_STATUS_STCCCH
```

A Common Command Code (CCC) message is being handled automatically

Definition at line 190 of file I3c_Ip_Types.h.

6.2.3.58 I3C_IP_SLAVE_STATUS_STMSG

```
#define I3C_IP_SLAVE_STATUS_STMSG
```

This slave is listening to the bus traffic or responding. For more details, please check the Reference Manual

Definition at line 191 of file I3c_Ip_Types.h.

6.2.3.59 I3C_IP_SLAVE_STATUS_STNOTSTOP

```
#define I3C_IP_SLAVE_STATUS_STNOTSTOP
```

The I3c bus is busy (STOP has not been detected yet)

Definition at line 192 of file I3c_Ip_Types.h.

6.2.3.60 I3C_IP_MASTER_MODE_IDLE

```
#define I3C_IP_MASTER_MODE_IDLE
```

Indicates the current master operation mode.

Bus stopped

Definition at line 204 of file I3c_Ip_Types.h.

6.2.3.61 I3C_IP_MASTER_MODE_SLAVE_REQUEST

```
#define I3C_IP_MASTER_MODE_SLAVE_REQUEST
```

Bus stopped but a slave is holding SDA low

Definition at line 205 of file I3c_Ip_Types.h.

6.2.3.62 I3C_IP_MASTER_MODE_MSG_SDR

```
#define I3C_IP_MASTER_MODE_MSG_SDR
```

In SDR Message mode

Definition at line 206 of file I3c_Ip_Types.h.

6.2.3.63 I3C_IP_MASTER_MODE_NORMAL_ACTIVE_SDR

```
#define I3C_IP_MASTER_MODE_NORMAL_ACTIVE_SDR
```

In Normal Active SDR mode

Definition at line 207 of file I3c_Ip_Types.h.

6.2.3.64 I3C_IP_MASTER_MODE_MSG_DDR

```
#define I3C_IP_MASTER_MODE_MSG_DDR
```

In DDR Message mode

Definition at line 208 of file I3c_Ip_Types.h.

6.2.3.65 I3C_IP_MASTER_MODE_DAA

```
#define I3C_IP_MASTER_MODE_DAA
```

In ENTDAA mode

Definition at line 209 of file I3c_Ip_Types.h.

6.2.3.66 I3C_IP_MASTER_MODE_IBI_ACK_NACK

```
#define I3C_IP_MASTER_MODE_IBI_ACK_NACK
```

Waiting on IBI ACK/NACK decision

Definition at line 210 of file I3c_Ip_Types.h.

6.2.3.67 I3C_IP_MASTER_MODE_IBI_RECEIVE

```
#define I3C_IP_MASTER_MODE_IBI_RECEIVE
```

Receiving an IBI

Definition at line 211 of file I3c_Ip_Types.h.

6.2.3.68 I3C_IP_CCC_ENEC_BROADCAST

```
#define I3C_IP_CCC_ENEC_BROADCAST
```

Common Command Codes. These defines should be used with I3c_SendCommonCommandCode() and I3c_↩ ReadCommonCommandCode(). For more details about each command, please see Mipi Basic Specifications v1.0 document.

Enable events

Definition at line 401 of file I3c_Ip_Types.h.

6.2.3.69 I3C_IP_CCC_DISEC_BROADCAST

```
#define I3C_IP_CCC_DISEC_BROADCAST
```

Disable events

Definition at line 402 of file I3c_Ip_Types.h.

6.2.3.70 I3C_IP_CCC_ENTAS0_BROADCAST

```
#define I3C_IP_CCC_ENTAS0_BROADCAST
```

Enter activity state 0

Definition at line 403 of file I3c_Ip_Types.h.

6.2.3.71 I3C_IP_CCC_ENTAS1_BROADCAST

```
#define I3C_IP_CCC_ENTAS1_BROADCAST
```

Enter activity state 1

Definition at line 404 of file I3c_Ip_Types.h.

6.2.3.72 I3C_IP_CCC_ENTAS2_BROADCAST

```
#define I3C_IP_CCC_ENTAS2_BROADCAST
```

Enter activity state 2

Definition at line 405 of file I3c_Ip_Types.h.

6.2.3.73 I3C_IP_CCC_ENTAS3_BROADCAST

```
#define I3C_IP_CCC_ENTAS3_BROADCAST
```

Enter activity state 3

Definition at line 406 of file I3c_Ip_Types.h.

6.2.3.74 I3C_IP_CCC_RSTDAA_BROADCAST

```
#define I3C_IP_CCC_RSTDAA_BROADCAST
```

Reset Dynamic Address Assignment

Definition at line 407 of file I3c_Ip_Types.h.

6.2.3.75 I3C_IP_CCC_ENTDAA_BROADCAST

```
#define I3C_IP_CCC_ENTDAA_BROADCAST
```

Enter Dynamic Address Assignment

Definition at line 408 of file I3c_Ip_Types.h.

6.2.3.76 I3C_IP_CCC_DEFSLVS_BROADCAST

```
#define I3C_IP_CCC_DEFSLVS_BROADCAST
```

Define list of slaves

Definition at line 409 of file I3c_Ip_Types.h.

6.2.3.77 I3C_IP_CCC_SETMWL_BROADCAST

```
#define I3C_IP_CCC_SETMWL_BROADCAST
```

Set maximum write length

Definition at line 410 of file I3c_Ip_Types.h.

6.2.3.78 I3C_IP_CCC_SETMRL_BROADCAST

```
#define I3C_IP_CCC_SETMRL_BROADCAST
```

Set maximum read length

Definition at line 411 of file I3c_Ip_Types.h.

6.2.3.79 I3C_IP_CCC_ENTTM_BROADCAST

```
#define I3C_IP_CCC_ENTTM_BROADCAST
```

Enter test mode

Definition at line 412 of file I3c_Ip_Types.h.

6.2.3.80 I3C_IP_CCC_ENDXFER_BROADCAST

```
#define I3C_IP_CCC_ENDXFER_BROADCAST
```

Provides the framework for Controllers and Targets to exchange set-up parameters for ending data transfers in supported HDR Modes. For HDR-DDR, it controls Data Transfer Ending Procedure. For more details, please check MIPI I3c Basic Specification document.

Definition at line 415 of file I3c_Ip_Types.h.

6.2.3.81 I3C_IP_CCC_ENTHDR0_BROADCAST

```
#define I3C_IP_CCC_ENTHDR0_BROADCAST
```

Enter HDR-DDR mode

Definition at line 416 of file I3c_Ip_Types.h.

6.2.3.82 I3C_IP_CCC_ENEC_DIRECT

```
#define I3C_IP_CCC_ENEC_DIRECT
```

Enable events

Definition at line 417 of file I3c_Ip_Types.h.

6.2.3.83 I3C_IP_CCC_DISEC_DIRECT

```
#define I3C_IP_CCC_DISEC_DIRECT
```

Disable events

Definition at line 418 of file I3c_Ip_Types.h.

6.2.3.84 I3C_IP_CCC_ENTAS0_DIRECT

```
#define I3C_IP_CCC_ENTAS0_DIRECT
```

Enter activity state 0

Definition at line 419 of file I3c_Ip_Types.h.

6.2.3.85 I3C_IP_CCC_ENTAS1_DIRECT

```
#define I3C_IP_CCC_ENTAS1_DIRECT
```

Enter activity state 1

Definition at line 420 of file I3c_Ip_Types.h.

6.2.3.86 I3C_IP_CCC_ENTAS2_DIRECT

```
#define I3C_IP_CCC_ENTAS2_DIRECT
```

Enter activity state 2

Definition at line 421 of file I3c_Ip_Types.h.

6.2.3.87 I3C_IP_CCC_ENTAS3_DIRECT

```
#define I3C_IP_CCC_ENTAS3_DIRECT
```

Enter activity state 3

Definition at line 422 of file I3c_Ip_Types.h.

6.2.3.88 I3C_IP_CCC_RSTDAA_DIRECT

```
#define I3C_IP_CCC_RSTDAA_DIRECT
```

Reset Dynamic Address Assignment

Definition at line 423 of file I3c_Ip_Types.h.

6.2.3.89 I3C_IP_CCC_SETDASA_DIRECT

```
#define I3C_IP_CCC_SETDASA_DIRECT
```

Set Dynamic Addres from Static Address

Definition at line 424 of file I3c_Ip_Types.h.

6.2.3.90 I3C_IP_CCC_SETNEWDA_DIRECT

```
#define I3C_IP_CCC_SETNEWDA_DIRECT
```

Set New Dynamic Addres

Definition at line 425 of file I3c_Ip_Types.h.

6.2.3.91 I3C_IP_CCC_SETMWL_DIRECT

```
#define I3C_IP_CCC_SETMWL_DIRECT
```

Set maximum write length

Definition at line 426 of file I3c_Ip_Types.h.

6.2.3.92 I3C_IP_CCC_SETMRL_DIRECT

```
#define I3C_IP_CCC_SETMRL_DIRECT
```

Set maximum read length

Definition at line 427 of file I3c_Ip_Types.h.

6.2.3.93 I3C_IP_CCC_GETMWL_DIRECT

```
#define I3C_IP_CCC_GETMWL_DIRECT
```

Get maximum write length

Definition at line 428 of file I3c_Ip_Types.h.

6.2.3.94 I3C_IP_CCC_GETMRL_DIRECT

```
#define I3C_IP_CCC_GETMRL_DIRECT
```

Get maximum read length

Definition at line 429 of file I3c_Ip_Types.h.

6.2.3.95 I3C_IP_CCC_GETPID_DIRECT

```
#define I3C_IP_CCC_GETPID_DIRECT
```

Get Provisional ID

Definition at line 430 of file I3c_Ip_Types.h.

6.2.3.96 I3C_IP_CCC_GETBCR_DIRECT

```
#define I3C_IP_CCC_GETBCR_DIRECT
```

Get Bus Characteristic Register

Definition at line 431 of file I3c_Ip_Types.h.

6.2.3.97 I3C_IP_CCC_GETDCR_DIRECT

```
#define I3C_IP_CCC_GETDCR_DIRECT
```

Get Device Characteristic Register

Definition at line 432 of file I3c_Ip_Types.h.

6.2.3.98 I3C_IP_CCC_GETSTATUS_DIRECT

```
#define I3C_IP_CCC_GETSTATUS_DIRECT
```

Get device status

Definition at line 433 of file I3c_Ip_Types.h.

6.2.3.99 I3C_IP_CCC_GETACCMST_DIRECT

```
#define I3C_IP_CCC_GETACCMST_DIRECT
```

Get accept mastership

Definition at line 434 of file I3c_Ip_Types.h.

6.2.3.100 I3C_IP_CCC_ENDXFER_DIRECT

```
#define I3C_IP_CCC_ENDXFER_DIRECT
```

Provides the framework for Controllers and Targets to exchange set-up parameters for ending data transfers in supported HDR Modes. For HDR-DDR, it controls Data Transfer Ending Procedure. For more details, please check MIPI I3c Basic Specification document.

Definition at line 437 of file I3c_Ip_Types.h.

6.2.3.101 I3C_IP_CCC_SETBRGTGT_DIRECT

```
#define I3C_IP_CCC_SETBRGTGT_DIRECT
```

Set bridge targets

Definition at line 438 of file I3c_Ip_Types.h.

6.2.3.102 I3C_IP_CCC_GETMXDS_DIRECT

```
#define I3C_IP_CCC_GETMXDS_DIRECT
```

Get maximum data speed

Definition at line 439 of file I3c_Ip_Types.h.

6.2.4 Types Reference

6.2.4.1 I3c_Ip_MasterCallbackType

```
typedef void(* I3c_Ip_MasterCallbackType) (I3c_Ip_MasterEventType Event)
```

Callback for all peripherals which supports I3C features for master mode.

Definition at line 464 of file I3c_Ip_Types.h.

6.2.4.2 I3c_Ip_SlaveCallbackType

```
typedef void(* I3c_Ip_SlaveCallbackType) (I3c_Ip_SlaveEventType Event)
```

Callback for all peripherals which supports I3C features for slave mode.

Definition at line 478 of file I3c_Ip_Types.h.

6.2.5 Enum Reference

6.2.5.1 I3c_Ip_MasterRequestType

```
enum I3c_Ip_MasterRequestType
```

Emits the requested operation when doing in pieces vs. by message.

Enumerator

I3C_IP_REQUEST_NONE	No request
I3C_IP_REQUEST_EMIT_START_ADDR	Emit START and ADDRESS on the bus
I3C_IP_REQUEST_EMIT_STOP	Emit STOP on the bus
I3C_IP_REQUEST_IBI_ACK_NACK	Manual IBI ACK or NACK
I3C_IP_REQUEST_PROCESS_DAA	Process DAA
I3C_IP_REQUEST_FORCE_EXIT	Emit Force Exit
I3C_IP_REQUEST_AUTO_IBI	Hold in a stopped state, but auto-emit START, I3C_IP_BROADCAST_ADDRESS to get an IBI

Definition at line 221 of file I3c_Ip_Types.h.

6.2.5.2 I3c_Ip_MasterEventType

enum I3c_Ip_MasterEventType

Events which can trigger I3C master callback.

Definition at line 236 of file I3c_Ip_Types.h.

6.2.5.3 I3c_Ip_MasterEnableType

enum I3c_Ip_MasterEnableType

Indicates if the master is enabled and what states are allowed for it (the master).

Enumerator

I3C_IP_MASTER_OFF	Master is not enabled
I3C_IP_MASTER_ON	Master is enabled
I3C_IP_MASTER_CAPABLE	Master capable, but operating as a slave

Definition at line 254 of file I3c_Ip_Types.h.

6.2.5.4 I3c_Ip_MasterHighKeepType

enum I3c_Ip_MasterHighKeepType

Indicates how High-Keeper is to be supported.

Enumerator

I3C_IP_MASTER_HIGH_KEEPER_NONE	Use PUR to hold SCL high
I3C_IP_MASTER_HIGH_KEEPER_WIRED_IN	Use pin_HK controls
I3C_IP_MASTER_PASSIVE_SDA	Passive on SDA: can Hi-Z for Bus Free and hold
I3C_IP_MASTER_PASSIVE_SDA_SCL	Passive on SDA and SCL: can Hi-Z both for Bus Free, and can Hi-Z SDA for hold

Definition at line 265 of file I3c_Ip_Types.h.

6.2.5.5 I3c_Ip_BusType

```
enum I3c_Ip_BusType
```

Bus type with EmitStartAddr.

Enumerator

I3C_IP_BUS_TYPE_I3C_SDR	I3C SDR mode
I3C_IP_BUS_TYPE_I2C	I2C mode
I3C_IP_BUS_TYPE_I3C_DDR	I3C HDR-DDR mode

Definition at line 277 of file I3c_Ip_Types.h.

6.2.5.6 I3c_Ip_DirectionType

```
enum I3c_Ip_DirectionType
```

Direction: send/receive.

Enumerator

I3C_IP_WRITE	Used to send data to a slave.
I3C_IP_READ	Used to receive data from a slave.

Definition at line 288 of file I3c_Ip_Types.h.

6.2.5.7 I3c_Ip_FifoTriggerLevelType

enum I3c_Ip_FifoTriggerLevelType

Watermark of Tx/RX INT/DMA trigger level.

Enumerator

I3C_IP_FIFO_TRIGGER_ON_EMPTY	Trigger on empty
I3C_IP_FIFO_TRIGGER_ON_ONE_QUARTER	Trigger on 1/4 full or less in Rx, and on 1/4 full or more in Tx
I3C_IP_FIFO_TRIGGER_ON_ONE_HALF	Trigger on 1/2 full or less in Rx, and on 1/4 full or more in Tx
I3C_IP_FIFO_TRIGGER_ON_ALMOST_FULL	Trigger on 1 less than full or less in Rx, and on 3/4 full or more in Tx

Definition at line 312 of file I3c_Ip_Types.h.

6.2.5.8 I3c_Ip_StatusType

enum I3c_Ip_StatusType

I3C specific error codes.

Enumerator

I3C_IP_STATUS_ERROR	An error occurred on the I3C Bus
I3C_IP_STATUS_BUSY	The I3C Bus is busy
I3C_IP_STATUS_TIMEOUT	The module has stalled too long in a frame
I3C_IP_STATUS_MAX_SLAVES_EXCEED	This status signals that the number of Slave Devices (on the bus) which need a Dynamic Address is larger than the number of available Dynamic Addresses.
I3C_IP_STATUS_IBI_WON	IBI/MR/HJ won the arbitration on a header address

Definition at line 324 of file I3c_Ip_Types.h.

6.2.5.9 I3c_Ip_TransferSizeType

enum I3c_Ip_TransferSizeType

Indicates size of the transfer message.

Enumerator

I3C_IP_TRANSFER_BYTES	Send/read in bytes
I3C_IP_TRANSFER_HALF_WORDS	Send/read in half-words

Definition at line 341 of file I3c_Ip_Types.h.

6.2.5.10 I3c_Ip_TransferType

enum I3c_Ip_TransferType

Type of I3C transfer (based on interrupts or DMA)

Enumerator

I3C_IP_USING_INTERRUPTS	The driver will use interrupts to perform a transfer

Definition at line 351 of file I3c_Ip_Types.h.

6.2.5.11 I3c_Ip_SlaveRequestType

enum I3c_Ip_SlaveRequestType

Indicates the current slave operation mode.

Enumerator

I3C_IP_SLAVE_NORMAL_MODE	Normal mode
I3C_IP_SLAVE_IBI_REQUEST	Start an IBI event
I3C_IP_SLAVE_MASTER_REQUEST	Start a MR or P2P event
I3C_IP_SLAVE_HOT_JOIN_REQUEST	Start a HJ event

Definition at line 363 of file I3c_Ip_Types.h.

6.2.5.12 I3c_Ip_SlaveEventType

enum I3c_Ip_SlaveEventType

Events which can trigger I3C slave callback.

Definition at line 375 of file I3c_Ip_Types.h.

6.2.6 Function Reference

6.2.6.1 I3c_Ip_MasterInit()

```
void I3c_Ip_MasterInit (
            const uint8 Instance,
            const I3c_Ip_MasterConfigType *const Config )
```

Initialize the I3C master module.

This function initializes the I3C master module.

Parameters

in	*Instance*	I3C instance
in	*Config*	Pointer to the configuration structure

6.2.6.2 I3c_Ip_MasterDeInit()

```
void I3c_Ip_MasterDeInit (
            const uint8 Instance )
```

Deinitialize the I3C master module.

This function deinitializes the master module. The driver can't be used until initialized again.

Parameters

in	*Instance*	I3C instance

6.2.6.3 I3c_Ip_MasterGetTransferStatus()

```
I3c_Ip_StatusType I3c_Ip_MasterGetTransferStatus (
            const uint8 Instance,
            uint32 *const BytesRemaining )
```

Return the current status of the I3C master transfer.

This function can be called during a non-blocking transmission to check the status of the transfer. In addition, if the transfer is still in progress, this functions returns the remaining bytes to be transferred, only if BytesRemaining pointer is different than NULL_PTR.

Parameters

in	*Instance*	I3C instance
out	*BytesRemaining*	Pointer to remaining bytes

6.2.6.4 I3c_Ip_MasterSendBlocking()

```
I3c_Ip_StatusType I3c_Ip_MasterSendBlocking (
            const uint8 Instance,
            const uint8 * TxBuffer,
            const uint32 TxBufferSize,
            const I3c_Ip_TransferConfigType *const TransferOption )
```

Perform a blocking send transaction on the I3C bus.

This function is used to send a block of data to the configured slave's address. It only returns when the transfer is complete.

Parameters

in	*Instance*	I3C instance
in	*TxBuffer*	Pointer to the Data buffer
in	*TxBufferSize*	Buffer size in bytes
in	*TransferOption*	Transfer options

6.2.6.5 I3c_Ip_MasterReceiveBlocking()

```
I3c_Ip_StatusType I3c_Ip_MasterReceiveBlocking (
            const uint8 Instance,
            uint8 * RxBuffer,
            const uint32 RxBufferSize,
            const I3c_Ip_TransferConfigType *const TransferOption )
```

Perform a blocking receive transaction on the I3C bus.

This function is used to receive a block of data from the configured slave's address. It only returns when the transfer is complete.

Parameters

in	*Instance*	I3C instance
in	*RxBuffer*	Pointer to the Data buffer
in	*RxBufferSize*	Buffer size in bytes
in	*TransferOption*	Transfer options

6.2.6.6 I3c_Ip_MasterSend()

```
I3c_Ip_StatusType I3c_Ip_MasterSend (
            const uint8 Instance,
            const uint8 * TxBuffer,
            const uint32 TxBufferSize,
            const I3c_Ip_TransferConfigType *const TransferOption )
```

Perform a non-blocking send transaction on the I3C bus.

This function is used to send a block of data to the configured slave's address. It returns immediately. The rest of the transfer is handled by the interrupt service routine. Use I3c_Ip_MasterGetTransferStatus() to check the progress of the transfer.

Parameters

in	*Instance*	I3C instance
in	*TxBuffer*	Pointer to the Data buffer
in	*TxBufferSize*	Buffer size in bytes
in	*TransferOption*	Transfer options

6.2.6.7 I3c_Ip_MasterReceive()

```
I3c_Ip_StatusType I3c_Ip_MasterReceive (
            const uint8 Instance,
            uint8 * RxBuffer,
            const uint32 RxBufferSize,
            const I3c_Ip_TransferConfigType *const TransferOption )
```

Perform a non-blocking receive transaction on the I3C bus.

This function is used to receive a block of data from the configured slave's address. It returns immediately. The rest of the transfer is handled by the interrupt service routine. Use I3c_Ip_MasterGetTransferStatus() to check the progress of the transfer.

Parameters

in	*Instance*	I3C instance
in	*RxBuffer*	Pointer to the Data buffer
in	*RxBufferSize*	Buffer size in bytes
in	*TransferOption*	Transfer options

6.2.6.8 I3c_Ip_SlaveInit()

```
void I3c_Ip_SlaveInit (
            const uint8 Instance,
            const I3c_Ip_SlaveConfigType *const Config )
```

Initialize the I3C slave module.

This function initializes the I3C slave module.

Parameters

in	*Instance*	I3C instance
in	*Config*	Pointer to the configuration structure

6.2.6.9 I3c_Ip_SlaveDeInit()

```
void I3c_Ip_SlaveDeInit (
            const uint8 Instance )
```

Deinitialize the I3C slave module.

This function deinitializes the slave module. The driver can't be used until initialized again.

Parameters

in	*Instance*	I3C instance

6.2.6.10 I3c_Ip_SlaveGetTransferStatus()

```
I3c_Ip_StatusType I3c_Ip_SlaveGetTransferStatus (
            const uint8 Instance,
            uint32 *const BytesRemaining )
```

Return the current status of the I3C slave transfer.

This function can be called during a non-blocking transmission to check the status of the transfer. In addition, if the transfer is still in progress, this functions returns the remaining bytes to be transferred, only if BytesRemaining pointer is different than NULL_PTR.

Parameters

in	*Instance*	I3C instance
in	*BytesRemaining*	Pointer to remaining bytes

6.2.6.11 I3c_Ip_MasterWriteOwnDynamicAddress()

```
void I3c_Ip_MasterWriteOwnDynamicAddress (
             const uint8 Instance,
             const uint8 Address,
             const boolean Valid )
```

The main master assigns its own Dynamic Address before handing off the mastership.

This function is used to set a Dynamic Address for the main master before handing off the mastership. This address should be written before switching to Slave mode and should not be changed once in Slave mode.

Parameters

in	*Instance*	I3C instance
in	*Address*	Dynamic Address for the main master to set before becoming a slave
in	*Valid*	If TRUE, a Dynamic Address is assigned

6.2.6.12 I3c_Ip_SlaveSetMapAddress()

```
void I3c_Ip_SlaveSetMapAddress (
             const uint8 Instance,
             const boolean AddressType,
             const uint16 Address,
             const boolean Nack,
             const boolean Enable )
```

Configure a slave which was the main master and just handed off the mastership.

This function is used to configure a slave at runtime, after it was the main master and handed off the mastership.

Parameters

in	*Instance*	I3C instance
in	*AddressType*	Address type: Dynamic Address or Static Address
in	*Address*	7-bit or 10-bit address
in	*Nack*	If TRUE, the slave will NACK all messages
in	*Enable*	If TRUE, the map entry is enabled.

6.2.6.13 I3c_Ip_MasterGetError()

```
uint32 I3c_Ip_MasterGetError (
            const uint8 Instance )
```

Returns the master error register.

This function is used to get the entire error/warning register for the current master. This may be called from the Error/Warning notification and dedicated defines should be used to filter the errors/warnings which occurred. Defines that can be used:

- I3C_IP_MASTER_ERROR_NACK
- I3C_IP_MASTER_ERROR_WRITE_ABORT
- I3C_IP_MASTER_ERROR_TERM
- I3C_IP_MASTER_ERROR_PARITY
- I3C_IP_MASTER_ERROR_CRC
- I3C_IP_MASTER_ERROR_OREAD
- I3C_IP_MASTER_ERROR_OWRITE
- I3C_IP_MASTER_ERROR_MSG
- I3C_IP_MASTER_ERROR_INVALID_REQUEST
- I3C_IP_MASTER_ERROR_TIMEOUT

Parameters

in	*Instance*	I3C instance

Returns

Master error register

6.2.6.14 I3c_Ip_SlaveGetError()

```
uint32 I3c_Ip_SlaveGetError (
            const uint8 Instance )
```

Returns the slave error register.

This function is used to get the entire error/warning register for the current slave. This may be called from the Error/Warning notification and dedicated defines should be used to filter the errors/warnings which occurred. Defines that can be used:

- I3C_IP_SLAVE_ERROR_RX_OVERRUN
- I3C_IP_SLAVE_ERROR_TX_UNDERRUN
- I3C_IP_SLAVE_ERROR_TX_UNDERRUN_NACK
- I3C_IP_SLAVE_ERROR_TERM
- I3C_IP_SLAVE_ERROR_INVALID_START
- I3C_IP_SLAVE_ERROR_SDR_PARITY
- I3C_IP_SLAVE_ERROR_HDR_PARITY_ERROR
- I3C_IP_SLAVE_ERROR_HDR_CRC
- I3C_IP_SLAVE_ERROR_S0S1
- I3C_IP_SLAVE_ERROR_OREAD
- I3C_IP_SLAVE_ERROR_OWRITE

Parameters

in	*Instance*	I3C instance

Returns

Slave error register

6.2.6.15 I3c_Ip_SlaveSetBuffer()

```
void I3c_Ip_SlaveSetBuffer (
             const uint8 Instance,
             uint8 * DataBuffer,
             const uint8 BufferSize )
```

Provide a buffer for the slave's data transfer.

This function is used to set the slave's buffer for the transfer data. It also set the default RX FIFO watermark. It could be called from the user callback provided at initialization.

Parameters

in	*Instance*	I3C instance
in	*DataBuffer*	Pointer to the data buffer
in	*BufferSize*	Data buffer size in bytes

6.2.6.16 I3c_Ip_SlavePrepareTxFifo()

```
void I3c_Ip_SlavePrepareTxFifo (
            const uint8 Instance )
```

Enables the slave Tx Send event if used in I2C mode.

This function is used to enable Slave TX interrupt. This is used to fill the Slave TX FIFO before master request. Note that before calling this function, I3c_Ip_SlaveSetBuffer() should be called.

Parameters

in	*Instance*	I3C instance

6.2.6.17 I3c_Ip_MasterSetBaudRate()

```
I3c_Ip_StatusType I3c_Ip_MasterSetBaudRate (
            const uint8 Instance,
            const uint32 FunctionalClock,
            const I3c_Ip_MasterBaudRateType * BaudRates,
            const I3c_Ip_BusType BusType )
```

Set the baud rate for all subsequent I3C communications.

This function is used to set the baud rate for all subsequent I3C communications. Using this function, the baud rates will be set as close as possible to the target ones. After calling this function, I3c_Ip_MasterGetBaudRate() shall be called to check which baud rates were actually set.

Parameters

in	*Instance*	I3C instance
in	*FunctionalClock*	Value of the Functional Clock (Hz)
in	*BaudRates*	Pointer to the Baud rates structure: PP, OD, I2C (Hz)
in	*BusType*	Type of the bus: I2C, SDR, DDR

6.2.6.18 I3c_Ip_MasterGetBaudRate()

```
void I3c_Ip_MasterGetBaudRate (
            const uint8 Instance,
            const uint32 FunctionalClock,
            I3c_Ip_MasterBaudRateType * BaudRates )
```

Get the baud rates of the I3C module.

This function is used to get the baud rates of the I3C module.

Parameters

in	*Instance*	I3C instance
in	*FunctionalClock*	Value of the Functional Clock (Hz)
out	*BaudRates*	Pointer to the Baud rates structure: PP, OD, I2C (Hz)

6.2.6.19 I3c_Ip_MasterGetStatus()

```
uint32 I3c_Ip_MasterGetStatus (
            const uint8 Instance )
```

Returns the master status register.

This function is used to get the status register for the current master. Dedicated defines can be used to filter and check each status bit field: I3C_IP_MASTER_STATUS For I3C_IP_MASTER_STATUS_IBIADDR, I3C_I↩ P_MASTER_STATUS_IBITYPE, and I3C_IP_MASTER_STATUS_STATE the following defines can be used to shift the masked values to get the actual value of the bit fields (they have more than 1 bit length):

- I3C_IP_MASTER_STATUS_IBIADDR_SHIFT
- I3C_IP_MASTER_STATUS_IBITYPE_SHIFT
- I3C_IP_MASTER_STATUS_STATE_SHIFT

Parameters

in	*Instance*	I3C instance

Returns

Master status register

6.2.6.20 I3c_Ip_SlaveGetStatus()

```
uint32 I3c_Ip_SlaveGetStatus (
            const uint8 Instance )
```

Returns the master status register.

This function is used to get the status register for the current slave device. Dedicated defines can be used to filter and check each status bit field: I3C_IP_SLAVE_STATUS For I3C_IP_SLAVE_STATUS_TIMECTRL, I3C_↩ IP_SLAVE_STATUS_ACTSTATE, and I3C_IP_SLAVE_STATUS_EVDET the following defines can be used to shift the masked values to get the actual value of the bit fields (they have more than 1 bit length):

- I3C_IP_SLAVE_STATUS_TIMECTRL_SHIFT
- I3C_IP_SLAVE_STATUS_ACTSTATE_SHIFT
- I3C_IP_SLAVE_STATUS_EVDET_SHIFT

Parameters

in	*Instance*	I3C instance

Returns

Slave status register

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, AltiVec, C–5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C–Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2022 NXP B.V.